

UMĚLÁ INTELIGENCE

V předchozích kapitolách jsme se zabývali algoritmy, které dostaly přesně popsany vstup a podle předem navrženého postupu vypočítaly výstup. V této kapitole se zaměříme na jinou situaci: pravidlo, které spojuje vstup s výstupem, neznáme, ale máme k dispozici data nebo zkušenosti, z nichž je možné vhodný model vytvořit. Právě tímto způsobem pracuje velká část současných systémů umělé inteligence.

Nejprve doplníme základy statistiky potřebné pro popis dat. Potom vymezíme strojové učení a jeho tři základní druhy. Podrobněji probereme lineární a logistickou regresi, klasifikaci pomocí k -NN a SVM a nakonec neuronové sítě včetně Hopfieldova modelu. Vedle matematického popisu budeme jednotlivé metody průběžně doprovázet jednoduchými implementacemi v jazyce Python.

1 Úvod do statistiky

Data sama o sobě nejsou hotovou informací. Číslo 45 000 například získá význam až ve chvíli, kdy víme, že jde o měsíční mzdu v korunách, známe období a skupinu lidí, ke které se vztahuje, a rozumíme způsobu jeho výpočtu. Statistika poskytuje jazyk a metody, jimiž lze větší množství dat uspořádat, shrnout a porovnat. Ve strojovém učení je tato role zásadní: model se učí právě z dat a jejich nevhodná interpretace se proto promítne i do jeho výsledků.

Definice 1.1. *Statistický znak* je sledovaná vlastnost objektů. Statistický soubor s n objekty a p sledovanými znaky budeme zapisovat jako matici

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}.$$

Hodnota x_{ij} je hodnota j -tého znaku u i -tého objektu.

Řádky matice tedy představují objekty a sloupce jednotlivé statistické znaky. Pokud sledujeme pouze jeden znak, zapisujeme statistický soubor jako vektor

$$\mathbf{X} = (x_1, x_2, \dots, x_n).$$

Tučné písmo je důležité: symbol $\mathbf{X}$ označuje celý statistický soubor, zatímco x_i označuje jednu jeho hodnotu.

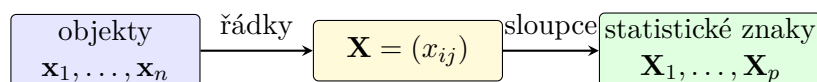


Figure 1: Dva rovnocenné pohledy na statistický soubor.

Pro stejně dlouhé statistické soubory

$$\mathbf{X} = (x_1, \dots, x_n), \quad \mathbf{Y} = (y_1, \dots, y_n)$$

a čísla $a, b \in \mathbb{R}$ budeme používat po složkách definované operace

$$a\mathbf{X} + b\mathbf{Y} = (ax_1 + by_1, \dots, ax_n + by_n).$$

Tento zápis využijeme například při posouvání, změně měřítka a standardizaci dat.

Souhrnná charakteristika nikdy nepopisuje každý objekt zvlášť. Průměrná mzda není mzdou „běžného“ člověka, nižší inflace neznamená pokles cen a výsledek za celou skupinu nemusí platit pro každou její část. Při interpretaci je vždy nutné znát kontext dat.

2 Charakteristiky polohy

Charakteristiky polohy nahrazují celý statistický soubor jedním číslem, které popisuje jeho typickou nebo střední hodnotu. Nejčastěji budeme používat aritmetický průměr a medián. Každá z těchto charakteristik zdůrazňuje jinou vlastnost dat.

2.1 Aritmetický a vážený průměr

Definice 2.1. *Aritmetický průměr* statistického souboru $\mathbf{X} = (x_1, \dots, x_n)$ je číslo

$$\bar{\mathbf{X}} = \frac{1}{n} \sum_{i=1}^n x_i.$$

Průměr zohledňuje všechny hodnoty a každé přisuzuje stejný význam. Nemusí být jednou z naměřených hodnot. Je také citlivý na odlehlá pozorování: jediná mimořádně vysoká nebo nízká hodnota jej může výrazně posunout.

Tvrzení 2.2. Pro statistické soubory $\mathbf{X}, \mathbf{Y}$ délky n a čísla $a, b \in \mathbb{R}$ platí

$$\sum_{i=1}^n (x_i - \bar{\mathbf{X}}) = 0 \quad \text{a} \quad \overline{a\mathbf{X} + b\mathbf{Y}} = a\bar{\mathbf{X}} + b\bar{\mathbf{Y}}.$$

Aritmetický průměr navíc minimalizuje součet čtvercových odchylek:

$$\bar{\mathbf{X}} = \arg \min_{c \in \mathbb{R}} \sum_{i=1}^n (x_i - c)^2.$$

Proof. První dvě rovnosti získáme přímým dosazením definice:

$$\sum_{i=1}^n (x_i - \bar{\mathbf{X}}) = \sum_{i=1}^n x_i - n\bar{\mathbf{X}} = 0$$

a

$$\overline{a\mathbf{X} + b\mathbf{Y}} = \frac{1}{n} \sum_{i=1}^n (ax_i + by_i) = a\bar{\mathbf{X}} + b\bar{\mathbf{Y}}.$$

Pro libovolné $c \in \mathbb{R}$ dále platí

$$\begin{aligned} \sum_{i=1}^n (x_i - c)^2 &= \sum_{i=1}^n ((x_i - \bar{\mathbf{X}}) + (\bar{\mathbf{X}} - c))^2 \\ &= \sum_{i=1}^n (x_i - \bar{\mathbf{X}})^2 + 2(\bar{\mathbf{X}} - c) \underbrace{\sum_{i=1}^n (x_i - \bar{\mathbf{X}})}_0 + n(\bar{\mathbf{X}} - c)^2. \end{aligned}$$

První člen nezávisí na c a poslední člen je nejmenší právě pro $c = \bar{\mathbf{X}}$. □

Ne vždy mají všechny hodnoty stejný význam. Máme-li vektor kladných vah $\mathbf{w} = (w_1, \dots, w_n)$, definujeme vážený průměr vztahem

$$\bar{\mathbf{X}}_{\mathbf{w}} = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}.$$

Po zavedení normalizovaných vah

$$\omega_i = \frac{w_i}{\sum_{j=1}^n w_j}$$

platí $\omega_i > 0$, $\sum_{i=1}^n \omega_i = 1$ a

$$\bar{\mathbf{X}}_{\mathbf{w}} = \sum_{i=1}^n \omega_i x_i.$$

Vážený průměr proto vždy leží mezi nejmenší a největší hodnotou souboru.

2.2 Medián

Seřazené hodnoty souboru budeme označovat

$$x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}.$$

Definice 2.3. *Medián* statistického souboru $\mathbf{X}$ je číslo

$$\text{med}(\mathbf{X}) = \begin{cases} x_{((n+1)/2)}, & \text{je-li } n \text{ liché,} \\ \frac{x_{(n/2)} + x_{(n/2+1)}}{2}, & \text{je-li } n \text{ sudé.} \end{cases}$$

Medián dělí uspořádaná data na dolní a horní polovinu. Na rozdíl od průměru jej jednotlivá odlehlá hodnota obvykle ovlivní jen málo. Pro soubor

$$\mathbf{X} = (2, 3, 4, 5, 100)$$

například dostáváme

$$\text{med}(\mathbf{X}) = 4, \quad \bar{\mathbf{X}} = 22,8.$$

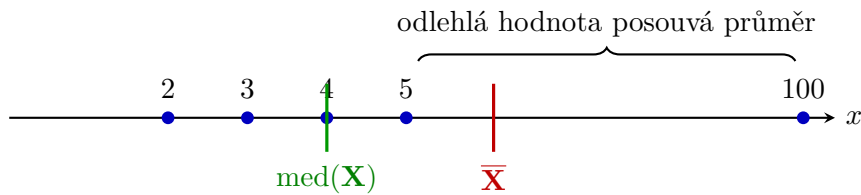


Figure 2: Odlehlá hodnota posune průměr podstatně více než medián.

Průměr je přirozený pro algebraické výpočty a pro metodu nejmenších čtverců. Medián je odolnější vůči odlehlým hodnotám. Volba charakteristiky proto závisí na povaze dat a na otázce, kterou chceme zodpovědět.

3 Charakteristiky variability

Dva statistické soubory mohou mít stejný průměr i medián, ale přesto se výrazně lišit. Hodnoty prvního mohou být soustředěny těsně kolem středu, zatímco hodnoty druhého mohou být rozptýleny po širokém intervalu. Charakteristiky variability popisují právě tuto rozptýlenost.

Definice 3.1. *Rozptýl* statistického souboru $\mathbf{X} = (x_1, \dots, x_n)$ definujeme vztahem

$$\text{var}(\mathbf{X}) = \sigma_{\mathbf{X}}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{\mathbf{X}})^2.$$

Směrodatná odchylka tohoto souboru je číslo

$$\sigma_{\mathbf{X}} = \sqrt{\text{var}(\mathbf{X})}.$$

Rozptýl je vždy nezáporný a je nulový právě tehdy, když jsou všechny hodnoty stejné. Protože pracuje se čtverci odchylek, jeho jednotka je druhou mocninou původní jednotky. Směrodatná odchylka tento problém odstraňuje a vyjadřuje typickou velikost odchylky ve stejné jednotce jako původní data.

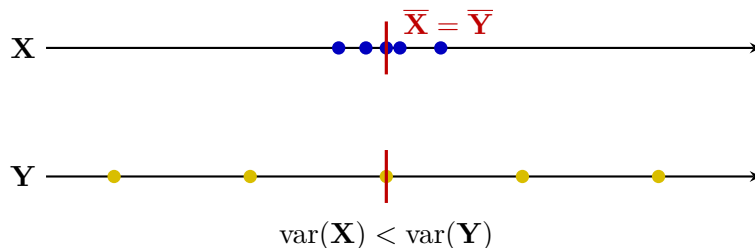


Figure 3: Soubory se stejným průměrem a různou variabilitou.

3.1 Cauchyova–Schwarzova nerovnost

Při práci s rozptylem a korelací budeme potřebovat následující základní nerovnost.

Věta 3.2 (Cauchyova–Schwarzova nerovnost). Pro libovolná reálná čísla $a_1, \dots, a_n, b_1, \dots, b_n$ platí

$$\left| \sum_{i=1}^n a_i b_i \right| \leq \sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}.$$

Rovnost nastává právě tehdy, když existuje $\lambda \in \mathbb{R}$ takové, že $b_i = \lambda a_i$ pro všechna i , nebo když jsou všechna a_i nulová.

Důkaz věty zde nebudeme uvádět. Geometricky jde o zobecnění skutečnosti, že absolutní hodnota skalárního součinu dvou vektorů nepřekročí součin jejich délek.

3.2 Vztah průměru, mediánu a směrodatné odchylky

Tvrzení 3.3. Pro každý statistický soubor $\mathbf{X} = (x_1, \dots, x_n)$ platí

$$\left| \bar{\mathbf{X}} - \text{med}(\mathbf{X}) \right| \leq \sigma_{\mathbf{X}}.$$

Proof. Označme $\mu = \bar{\mathbf{X}}$ a $m = \text{med}(\mathbf{X})$. Stačí vyřešit případ $\mu \geq m$; případ $\mu < m$ dostaneme stejným postupem po změně znamének.

Položme

$$A = \{i : x_i \leq m\}, \quad k = |A|.$$

Z definice mediánu plyne $k \geq n/2$. Pro $i \in A$ definujme $a_i = \mu - x_i$. Potom $a_i \geq \mu - m$. Označíme-li

$$S = \sum_{i \in A} a_i,$$

dostáváme $S \geq k(\mu - m)$. Součet všech odchylek od průměru je nulový, a proto

$$\sum_{i \notin A} (x_i - \mu) = S.$$

Z Cauchyovy–Schwarzovy nerovnosti použité zvlášť na obě skupiny plyne

$$\sum_{i \in A} (x_i - \mu)^2 \geq \frac{S^2}{k} \quad \text{a} \quad \sum_{i \notin A} (x_i - \mu)^2 \geq \frac{S^2}{n - k}.$$

Je-li $k = n$, jsou všechny hodnoty nejvýše μ , a nulový součet odchylek vynutí $x_i = \mu = m$; tvrzení je okamžité. Pro $k < n$ sečtením nerovností získáme

$$\begin{aligned} \sigma_{\mathbf{X}}^2 &= \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2 \\ &\geq \frac{1}{n} \left(\frac{S^2}{k} + \frac{S^2}{n - k} \right) = \frac{S^2}{k(n - k)} \\ &\geq \frac{k}{n - k} (\mu - m)^2 \geq (\mu - m)^2. \end{aligned}$$

Poslední nerovnost plyne z $k \geq n/2$. Po odmocnění tedy $\mu - m \leq \sigma_{\mathbf{X}}$. □

Tvrzení neříká, že průměr a medián musí být blízko v absolutním měřítku. Říká, že jejich vzdálenost nemůže překročit směrodatnou odchylku daného souboru.

4 Standardizace dat

Různé statistické znaky mohou používat odlišné jednotky a měřítka. Věk člověka se může pohybovat v desítkách let, zatímco jeho roční příjem ve statisících korun. Metoda založená na vzdálenosti nebo velikosti koeficientů pak může bez úpravy dat přikládat příjmu nepřiměřeně velký význam.

Definice 4.1. Nechť $\mathbf{X} = (x_1, \dots, x_n)$ a $\sigma_{\mathbf{X}} > 0$. *Standardizovanou hodnotou* x_i rozumíme číslo

$$z_i = \frac{x_i - \bar{\mathbf{X}}}{\sigma_{\mathbf{X}}}.$$

Standardizovaný statistický soubor značíme

$$\mathbf{Z} = (z_1, \dots, z_n).$$

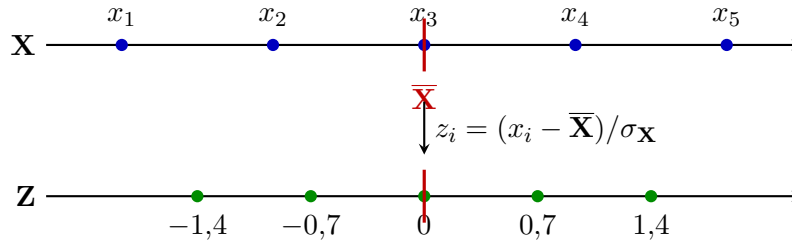


Figure 4: Standardizace posune průměr do nuly a změni měřítko.

Hodnota z_i udává, o kolik směrodatných odchylek leží x_i nad nebo pod průměrem. Kladné hodnoty leží nad průměrem, záporné pod ním a hodnota 0 odpovídá přímo průměru.

Tvrzení 4.2. Pro standardizovaný soubor $\mathbf{Z}$ platí

$$\bar{\mathbf{Z}} = 0, \quad \text{var}(\mathbf{Z}) = 1, \quad \sigma_{\mathbf{Z}} = 1.$$

Proof. Z vlastností aritmetického průměru dostáváme

$$\bar{\mathbf{Z}} = \frac{1}{n} \sum_{i=1}^n \frac{x_i - \bar{\mathbf{X}}}{\sigma_{\mathbf{X}}} = \frac{1}{n\sigma_{\mathbf{X}}} \sum_{i=1}^n (x_i - \bar{\mathbf{X}}) = 0.$$

Proto

$$\begin{aligned} \text{var}(\mathbf{Z}) &= \frac{1}{n} \sum_{i=1}^n z_i^2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{\mathbf{X}}}{\sigma_{\mathbf{X}}} \right)^2 \\ &= \frac{\text{var}(\mathbf{X})}{\sigma_{\mathbf{X}}^2} = 1. \end{aligned}$$

Odtud $\sigma_{\mathbf{Z}} = \sqrt{\text{var}(\mathbf{Z})} = 1$. □

Průměr a směrodatnou odchylku je nutné vypočítat pouze z trénovacích dat. Stejně dvě hodnoty potom použijeme i pro validaci, testování a nové objekty. Kdybychom do výpočtu zahrnuli testovací data, předali bychom modelu informaci, kterou při skutečném nasazení nemá mít.

5 Min-max normalizace dat

Standardizace popisuje hodnoty pomocí vzdálenosti od průměru. Jinou možností je převést nejmenší hodnotu na 0, největší na 1 a ostatní mezi ně lineárně rozmístit.

Definice 5.1. Nechť $\mathbf{X} = (x_1, \dots, x_n)$ a

$$m = \min_{1 \leq i \leq n} x_i, \quad M = \max_{1 \leq i \leq n} x_i.$$

Je-li $M > m$, definujeme *min-max normalizované hodnoty*

$$u_i = \frac{x_i - m}{M - m}$$

a normalizovaný soubor $\mathbf{U} = (u_1, \dots, u_n)$.

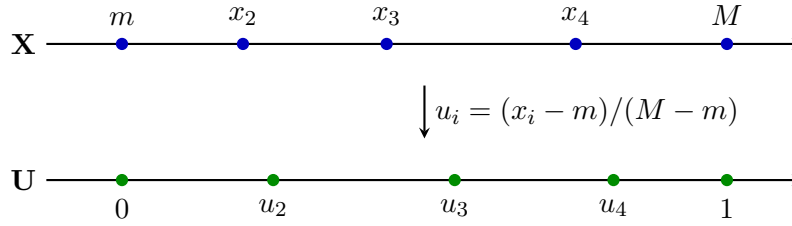


Figure 5: Min-max normalizace převádí rozsah dat na interval $\langle 0, 1 \rangle$.

Pro každé i platí $0 \leq u_i \leq 1$. Nejmenší původní hodnota se zobrazí na 0, největší na 1 a pořadí hodnot se nezmění. Jde totiž o rostoucí lineární transformaci.

Pro soubor teplot

$$\mathbf{T} = (18, 20, 20, 21, 26)$$

je $m = 18$, $M = 26$, a tedy

$$\mathbf{U} = (0; 0,25; 0,25; 0,375; 1).$$

Standardizace i min-max normalizace odstraňují fyzikální jednotku a pomáhají porovnávat různé znaky. Liší se však významem výsledných hodnot. Standardizace zachovává informaci o vzdálenosti od průměru ve směrodatných odchylkách a není omezená na pevný interval. Min-max normalizace zaručí interval $\langle 0, 1 \rangle$, ale silně závisí na krajních hodnotách. Pro metody založené na vzdálenosti, například k -NN nebo k -means, jsou použitelné obě; vhodnější volba závisí na rozdělení dat a na výskytu odlehlých hodnot.

Stejně jako u standardizace určujeme m a M pouze z trénovacích dat. Nová hodnota může ležet mimo původní rozsah, a její normalizovaná hodnota proto může být menší než 0 nebo větší než 1.

6 Korelace

Mějme dva stejně dlouhé statistické soubory

$$\mathbf{X} = (x_1, \dots, x_n), \quad \mathbf{Y} = (y_1, \dots, y_n).$$

Dvojice (x_i, y_i) popisuje dva znaky téhož i -tého objektu. Bodový graf těchto dvojic často naznačí, zda se s růstem jednoho znaku mění také druhý.

Definice 6.1. Kovariance souborů $\mathbf{X}$ a $\mathbf{Y}$ je číslo

$$\text{cov}(\mathbf{X}, \mathbf{Y}) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{\mathbf{X}})(y_i - \bar{\mathbf{Y}}).$$

Kladná kovariance obvykle znamená, že nadprůměrné hodnoty $\mathbf{X}$ doprovázejí nadprůměrné hodnoty $\mathbf{Y}$. Záporná kovariance odpovídá opačnému směru. Její velikost však závisí na jednotkách obou znaků, a proto ji nelze přímo porovnávat mezi různými dvojicemi dat.

Definice 6.2. Necht $\sigma_{\mathbf{X}} > 0$ a $\sigma_{\mathbf{Y}} > 0$. Korelační koeficient souborů $\mathbf{X}, \mathbf{Y}$ definujeme vztahem

$$\rho_{\mathbf{XY}} = \frac{\text{cov}(\mathbf{X}, \mathbf{Y})}{\sigma_{\mathbf{X}} \sigma_{\mathbf{Y}}}.$$

Ekvivalentně lze psát

$$\rho_{\mathbf{XY}} = \frac{\sum_{i=1}^n (x_i - \bar{\mathbf{X}})(y_i - \bar{\mathbf{Y}})}{\sqrt{\sum_{i=1}^n (x_i - \bar{\mathbf{X}})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{\mathbf{Y}})^2}}.$$

Korelace je tedy kovariance po odstranění měřítka obou znaků.

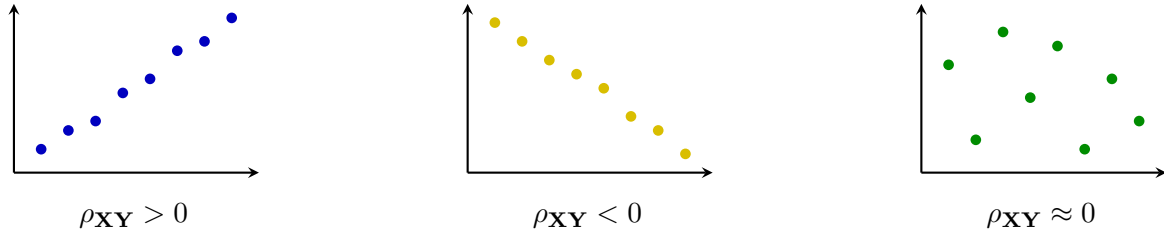


Figure 6: Kladná, záporná a nulová lineární korelace.

Tvrzení 6.3. Pro korelační koeficient souborů $\mathbf{X}, \mathbf{Y}$ platí:

1. $-1 \leq \rho_{\mathbf{XY}} \leq 1$;
2. $|\rho_{\mathbf{XY}}| = 1$ právě tehdy, když

$$\mathbf{Y} = \alpha \mathbf{X} + \beta$$

pro vhodná $\alpha, \beta \in \mathbb{R}$ s $\alpha \neq 0$. Přitom $\rho_{\mathbf{XY}} = 1$ pro $\alpha > 0$ a $\rho_{\mathbf{XY}} = -1$ pro $\alpha < 0$.

Proof. Položme

$$a_i = x_i - \bar{\mathbf{X}}, \quad b_i = y_i - \bar{\mathbf{Y}}.$$

Podle Cauchyovy–Schwarzovy nerovnosti platí

$$\left| \sum_{i=1}^n a_i b_i \right| \leq \sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}.$$

Protože oba výrazy pod odmocninou jsou kladné, můžeme jimi nerovnost vydělit a dostaneme

$$|\rho_{\mathbf{XY}}| \leq 1.$$

Rovnost nastane právě tehdy, když jsou vektory odchylek lineárně závislé, tedy když existuje $\alpha \neq 0$ takové, že

$$y_i - \bar{\mathbf{Y}} = \alpha(x_i - \bar{\mathbf{X}})$$

pro všechna i . Po úpravě

$$y_i = \alpha x_i + \underbrace{\bar{\mathbf{Y}} - \alpha \bar{\mathbf{X}}}_{\beta},$$

a tedy $\mathbf{Y} = \alpha \mathbf{X} + \beta$. Obráceně, z této rovnosti ihned plyne lineární závislost obou vektorů odchylek, a tedy rovnost v Cauchyově–Schwarzově nerovnosti.

Nakonec

$$\text{cov}(\mathbf{X}, \mathbf{Y}) = \text{cov}(\mathbf{X}, \alpha \mathbf{X} + \beta) = \alpha \text{var}(\mathbf{X}),$$

takže znaménko korelace je znaménkem α . □

Korelace měří pouze lineární vztah. Hodnota $\rho_{\mathbf{XY}} = 0$ nevylučuje nelineární závislost. Korelace také sama o sobě nedokazuje příčinnou souvislost; společný pohyb obou znaků může být způsoben třetí proměnnou nebo náhodou.

7 Úvod do strojového učení

Pojem umělá inteligence nemá jedinou všeobecně přijímanou definici. V literatuře se setkáme se čtyřmi základními pohledy: systém může napodobovat lidské myšlení, napodobovat lidské jednání, usilovat o správné racionální uvažování nebo jednat jako racionální agent. Každý pohled klade důraz na jinou otázku.

Napodobování lidského jednání zachycuje Turingův test. Člověk písemně komunikuje se dvěma skrytými účastníky, z nichž jeden je člověk a druhý počítač. Jestliže je podle odpovědí nedokáže spolehlivě rozlišit, počítač v testu uspěl. Takový výsledek však přímo neříká, že počítač přemýšlí stejným způsobem jako člověk; hodnotí pouze pozorované chování.

Pro technický návrh systémů je užitečný pohled inteligentního agenta. *Agent* pozoruje své prostředí, volí akce a jejich prostřednictvím prostředí ovlivňuje. Za racionální považujeme takového agenta, který s ohledem na dostupné informace volí akce vedoucí k co nejlepšímu výsledku. Racionalita neznamená neomylnost: agent před rozhodnutím nemusí znát budoucnost a může mít omezený čas i výpočetní prostředky.

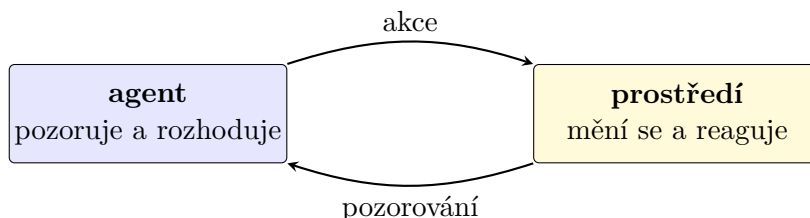


Figure 7: Agent pozoruje prostředí a ovlivňuje je svými akcemi.

Strojové učení je pouze jednou částí umělé inteligence. Vedle něj sem patří například plánování, reprezentace znalostí, logické usuzování, zpracování přirozeného jazyka, počítačové vidění nebo robotika. Učení je však důležité proto, že agentovi umožňuje přizpůsobovat chování novým datům a zkušenostem, aniž by programátor musel předem zapsat pravidlo pro každou možnou situaci.

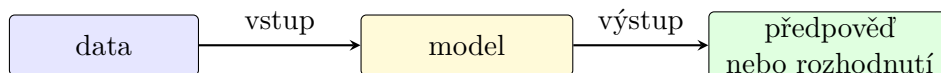


Figure 8: Data se prostřednictvím modelu mění v předpověď nebo rozhodnutí.

Umělá inteligence není synonymem neuronových sítí ani strojového učení. Strojové učení je skupina metod, které vytvářejí nebo upravují model na základě dat či zkušeností.

Poznámka 7.1. Alan Turing formuloval svůj napodobovací test v článku z roku 1950. Samotné označení *artificial intelligence* navrhl John McCarthy pro letní seminář v Dartmouthu roku 1956; tato událost se obvykle považuje za symbolický počátek umělé inteligence jako samostatného oboru.

8 Bližší vymezení strojového učení

U běžného algoritmu stanoví programátor přesný postup výpočtu. U strojového učení místo toho určíme třídu modelů, způsob měření chyby a postup, kterým se parametry modelu upravují podle dat. Výsledné konkrétní pravidlo tedy nevzniká pouze přímým zápisem programu, ale také trénováním.

Učení lze popsat pomocí tří složek:

- **úloha** říká, co má model provádět, například předpovídat cenu nebo rozpoznávat spam;
- **zkušenost** představuje data nebo interakce, z nichž model čerpá;
- **měřítka úspěchu** určuje, podle čeho poznáme zlepšení modelu.

O učení hovoříme tehdy, když se s rostoucí zkušeností zlepšuje výkon modelu v dané úloze podle zvoleného měřítka.

8.1 Trénování a použití modelu

Trénováním rozumíme hledání parametrů modelu na základě dat. Při následném *použití* jsou parametry pevné a model vypočítává výstupy pro nové vstupy. Tyto dvě fáze je nutné odlišovat: náročné hledání parametrů může probíhat jednou, zatímco hotový model potom používáme mnohokrát.

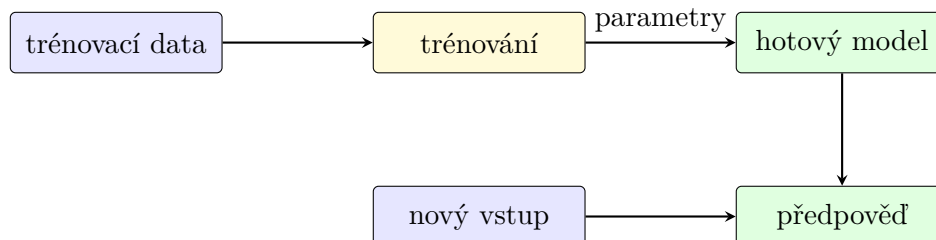


Figure 9: Odlišení trénování modelu od jeho následného použití.

Pro kontrolu schopnosti zobecnit rozdělujeme dostupná data na tři části:

- **trénovací data** slouží k určení parametrů modelu;
- **validační data** slouží k volbě nastavení, například hodnoty k , rozhodovacího prahu nebo počtu epoch;
- **testovací data** použijeme až na konci pro nezávislé závěrečné vyhodnocení.

Testovací data nesmějí ovlivnit trénování ani ladění. Opakované rozhodování podle výsledku na testovací množině by z ní postupně udělalo další validační množinu.

8.2 Základní členění

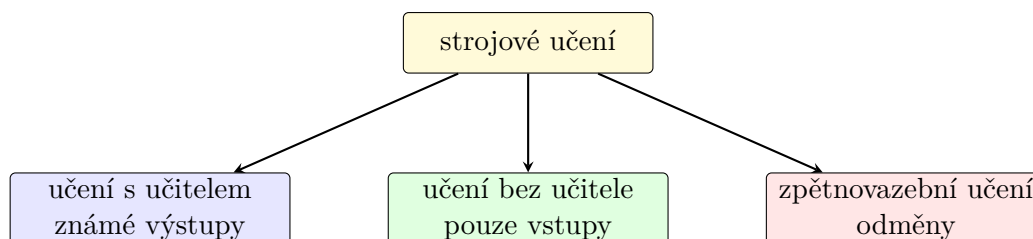


Figure 10: Tři základní druhy strojového učení.

- Při **učení s učitelem** známe u trénovacích objektů požadované výstupy.
- Při **učení bez učitele** výstupy nebo třídy předem dány nejsou a hledáme strukturu ve vstupních datech.

- Při **zpětnovazebním učení** agent volí akce, pozoruje jejich následky a učí se z odměn.

Toto členění neurčuje konkrétní typ matematického modelu. Neuronovou síť lze například trénovat s učitelem pro klasifikaci, bez učitele pro hledání reprezentace i zpětnovazebně jako součást agenta. Rozhodující je druh dostupné zkušenosti a způsob, jakým model dostává informaci o úspěchu.

V praxi se jednotlivé druhy mohou kombinovat. Model lze nejprve naučit bez učitele na velkém množství neoznačených dat a potom jej doladit s učitelem na menší označené množině. Agent ve zpětnovazebním učení může zase používat model natrénovaný s učitelem k rozpoznávání stavu prostředí.

Nízká chyba na trénovacích datech sama o sobě nestačí. Smyslem modelu je pracovat s novými daty. Pokud si model pouze zapamatuje zvláštnosti trénovací množiny, dochází k přeučení.

9 Učení s učitelem

Při učení s učitelem máme pro každý trénovací objekt k dispozici jak vstupní znaky, tak správný výstup. Necht

$$\mathbf{X}_1, \dots, \mathbf{X}_p$$

jsou vstupní statistické soubory a

$$\mathbf{Y} = (y_1, \dots, y_n)$$

je výstupní statistický soubor. i -tý objekt popisuje vektor

$$\mathbf{x}_i = (x_{i1}, \dots, x_{ip}),$$

a trénovací množinu tedy můžeme zapsat

$$D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}.$$

Model h vytváří pro nový objekt $\mathbf{x}$ předpověď $h(\mathbf{x})$. Při trénování porovnáváme předpovědi na trénovacích objektech se známými hodnotami y_i a upravujeme model tak, aby se zvolená chyba zmenšovala.

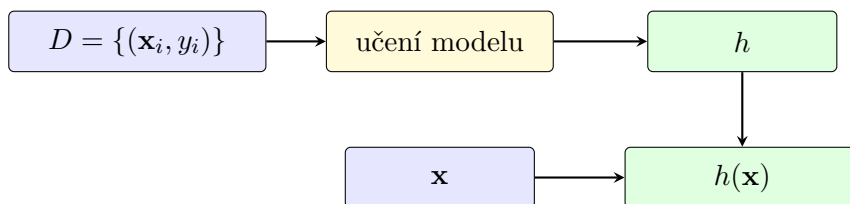


Figure 11: Základní postup učení s učitelem.

Podle povahy výstupu rozlišujeme zejména dvě úlohy:

- **regrese** předpovídá číselnou hodnotu, například cenu bytu, spotřebu energie nebo dobu jízdy;
- **klasifikace** vybírá z konečné množiny tříd, například spam a běžnou zprávu nebo jednu z několika kategorií obrazu.

Dobře fungující model musí zachytit obecný vztah mezi vstupy a výstupem. Příliš jednoduchý model důležitou strukturu nevystihne; příliš složitý model může místo ní zachytit šum. Volba třídy modelů proto určuje kompromis mezi schopností přizpůsobit se datům a schopností zobecnit.

Slovo „učitel“ neznamená, že model vede člověk krok za krokem. Učitelem je zde známý výstup y_i , s nímž lze porovnat předpověď modelu.

10 Učení bez učitele

Při učení bez učitele známe pouze vstupní objekty

$$\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^p,$$

ale nemáme k nim připojený výstupní soubor $\mathbf{Y}$. Cílem proto není napodobit známé správné odpovědi, nýbrž objevit užitečnou strukturu dat. Může jít o shluky podobných objektů, odlehlá pozorování, menší počet skrytých znaků nebo často se opakující vztahy.

Na rozdíl od učení s učitelem zde neexistuje jediná předem známá správná odpověď, s níž bychom mohli výsledek jednoduše porovnat. Dvě různá rozdělení mohou být smysluplná pro dva různé účely: zákazníky lze například seskupovat podle nákupního chování, podle věku nebo podle místa bydliště. Před interpretací výsledku proto musíme rozumět použitým znakům a otázce, kterou datům klademe.

Značení. Pro vektor $\mathbf{u} = (u_1, \dots, u_p) \in \mathbb{R}^p$ budeme symbolem

$$\|\mathbf{u}\| = \sqrt{\sum_{r=1}^p u_r^2}$$

označovat jeho *eukleidovskou normu*, tedy jeho délku. Vzdálenost vektorů $\mathbf{u}$ a $\mathbf{v}$ je potom $\|\mathbf{u} - \mathbf{v}\|$. Dvojitě svislé čáry tedy v této kapitole vždy vyjadřují délku vektoru, nikoli absolutní hodnotu čísla.

10.1 Shlukování a algoritmus k -means

Shlukování rozděluje objekty do skupin tak, aby si objekty uvnitř stejného shluku byly podobné a objekty z různých shluků odlišné. Podobnost obvykle vyjadřujeme vzdáleností. Výsledek proto vždy závisí na zvoleném popisu objektů, jejich měřítku a konkrétní vzdálenosti.

Algoritmus k -means hledá k shluků

$$C_1, \dots, C_k$$

a jejich středy

$$\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k.$$

Přiřazení objektu $\mathbf{x}_i$ budeme značit $c(i) \in \{1, \dots, k\}$. Shluk číslo j je tedy množina

$$C_j = \{\mathbf{x}_i : c(i) = j\}.$$

Je-li $C_j \neq \emptyset$, jeho střed definujeme po složkách jako aritmetický průměr všech vektorů ve shluku:

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i.$$

Například první souřadnice μ_j je průměrem prvních souřadnic objektů z C_j . Pokud během výpočtu některý shluk zůstane prázdný, průměr není definován; jednoduchá implementace proto ponechá jeho dosavadní střed, případně lze zvolit nový střed z dat.

Pro eukleidovskou vzdálenost probíhá opakování dvou kroků:

1. každý objekt přiřadíme k nejbližšímu středu;
2. každý střed nahradíme aritmetickým průměrem objektů přiřazeného shluku.

Algoritmus 10.1: K-MEANS

Vstup: Objekty $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^p$, počet shluků k a počáteční středy $\mu_1, \dots, \mu_k$

dokud některé přiřazení nebo střed se změní **opakuji**

pro každý objekt $\mathbf{x}_i$ **opakuji**

$c(i) \leftarrow \arg \min_{j \in \{1, \dots, k\}} \|\mathbf{x}_i - \mu_j\|$

pro $j \leftarrow 1, 2, \dots, k$ **opakuji**

$C_j \leftarrow \{\mathbf{x}_i : c(i) = j\}$

pokud $C_j \neq \emptyset$ **pak**

$\mu_j \leftarrow \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i$

Výstup: Shluky $C_1, \dots, C_k$ a jejich středy $\mu_1, \dots, \mu_k$

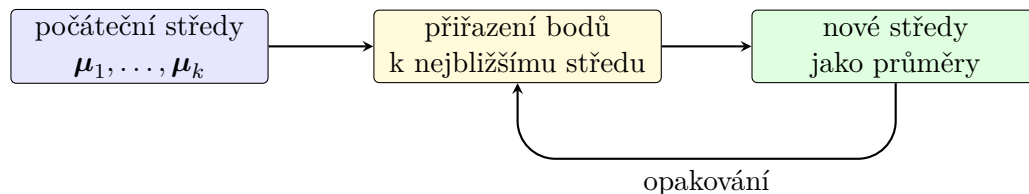


Figure 12: Jedna iterace algoritmu k -means.

Algoritmus se snaží zmenšovat součet čtvercových vzdáleností

$$J = \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \mu_j\|^2.$$

Pro pevně zvolené shluky je nejlepším středem jejich aritmetický průměr; jde o vícerozměrnou obdobu minimalizační vlastnosti průměru. Přiřazovací krok zase pro pevné středy volí pro každý objekt nejmenší možný příspěvek do J . Hodnota J se tedy v každém kroku nezvětší. Protože existuje jen konečně mnoho přiřazení n objektů do k shluků, algoritmus se po konečném počtu změn zastaví.

Výsledné rozdělení nemusí být nejlepší ze všech možností. Závisí na počátečních střezech, předem zvoleném počtu k a na měřítku znaků. Algoritmus je vhodný hlavně pro přibližně kulové a podobně velké shluky.

Poznámka 10.1. Název k -means zavedl James MacQueen roku 1967. Velmi podobný iterační postup popsal Stuart Lloyd už roku 1957 pro kvantování signálu; jeho práce však vyšla tiskem až roku 1982. Proto se algoritmus někdy označuje také jako Lloydův algoritmus.

10.2 Jednoduchá implementace

Program 10.1 zachycuje přímo oba kroky algoritmu. Pro jednoduchost pracuje s dvourozměrnými body, výpočet vzdálenosti je však možné stejným způsobem rozšířit na libovolné p .

Počáteční středy zde tvoří prvních k bodů. Taková volba je snadno čitelná, ale v praxi může vést k nevhodnému lokálnímu minimu. Běžnou alternativou je několik náhodných spuštění nebo inicializace k -means++, která vybírá nové středy s ohledem na jejich vzdálenost od středů již zvolených.

Program 10.1: Jednoduchá implementace algoritmu k -means.

```
1 from math import dist
2
3
4 def mean(points):
5     dimension = len(points[0])
6     return tuple(
7         sum(point[j] for point in points) / len(points)
8         for j in range(dimension)
9     )
10
11
12 def assign(points, centers):
13     return [
14         min(range(len(centers)), key=lambda j: dist(point, centers[j]))
15         for point in points
16     ]
17
18
19 def update(points, labels, centers):
20     new_centers = []
21     for j, old_center in enumerate(centers):
22         cluster = [
23             point for point, label in zip(points, labels)
24             if label == j
25         ]
26         new_centers.append(mean(cluster) if cluster else old_center)
27     return new_centers
28
29
30 def kmeans(points, k, max_iterations=100, tolerance=1e-6):
31     if not 1 <= k <= len(points):
32         raise ValueError("k must be between 1 and the number of points")
33
34     centers = list(points[:k])
35     for _ in range(max_iterations):
36         labels = assign(points, centers)
37         new_centers = update(points, labels, centers)
38         movement = max(
39             dist(old, new)
40             for old, new in zip(centers, new_centers)
41         )
42         centers = new_centers
43         if movement <= tolerance:
44             break
45
46     return labels, centers
47
```

```

48
49 if __name__ == "__main__":
50     data = [(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)]
51     labels, centers = kmeans(data, k=2)
52     print("labels:", labels)
53     print("centers:", centers)

```

Funkce `assign` vyhledá nejbližší střed, zatímco `update` z přiřazených bodů vytvoří nové průměry. Iterace skončí, jakmile se středy nezmění více než o zadanou toleranci.

Výstupem nejsou názvy tříd, ale pouze čísla shluků. Číslo samotné nemá věcný význam: při jiném spuštění si mohou dva shluky svá čísla prohodit, aniž by se změnilo jejich rozdělení. Význam skupin proto vždy určujeme až následnou interpretací jejich středů a typických objektů.

11 Zpětnovazební učení

Při zpětnovazebním učení agent nedostává pro každý stav předem předepsanou správnou akci. Musí jednat, sledovat následky a postupně zjišťovat, které volby vedou k dobrému dlouhodobému výsledku.

Mějme množinu stavů $\mathcal{S}$ a pro každý stav $s \in \mathcal{S}$ množinu přípustných akcí $\mathcal{A}(s)$. V čase t se agent nachází ve stavu s_t , zvolí akci $a_t \in \mathcal{A}(s_t)$ a prostředí mu vrátí odměnu $r_{t+1} \in \mathbb{R}$ a nový stav s_{t+1} . Jednu zkušenost tedy popisuje čtveřice

$$(s_t, a_t, r_{t+1}, s_{t+1}).$$

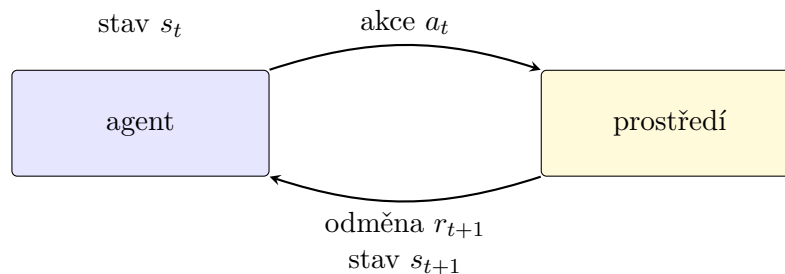


Figure 13: Cyklus interakce agenta s prostředím.

Agent obvykle nechce získat pouze nejvyšší okamžitou odměnu. Zajímá ho diskontovaný součet budoucích odměn

$$G(t) = \sum_{i=0}^{\infty} \gamma^i r_{t+i+1}, \quad 0 \leq \gamma < 1.$$

Parametr γ určuje význam budoucnosti. Pro γ blízké nule převažuje okamžitý zisk; pro γ blízké jedné agent více zohledňuje vzdálenější následky.

Politika π přiřazuje stavům akce, tedy $\pi(s) = a$. Cílem je naučit se politiku, která vede k co nejvyššímu dlouhodobému zisku. Agent přitom řeší dilema mezi využíváním dosud nejlepší známé akce a průzkumem jiných možností, které se mohou ukázat jako výhodnější.

11.1 Q-learning

Algoritmus Q-learning uchovává pro každou přípustnou dvojici (s, a) hodnotu $Q(s, a)$, která vyjadřuje, jak výhodné se zatím jeví provést akci a ve stavu s . Po zkušenosti $(s_t, a_t, r_{t+1}, s_{t+1})$ provede aktualizaci

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right],$$

kde $\alpha \in (0, 1]$ je rychlost učení. Výraz v hranaté závorce porovnává dosavadní odhad s novou informací: okamžitou odměnou a odhadovanou hodnotou nejlepšího pokračování.

Aktualizace nemění celou tabulku, ale pouze položku odpovídající právě provedené akci. Je-li nově pozorovaný výsledek lepší než dosavadní odhad, hodnota $Q(s_t, a_t)$ vzroste; je-li horší, klesne. Parametr α určuje, jak silně jediná zkušenost přepíše dřívější poznatky, zatímco γ určuje, jak velkou část hodnoty připisujeme budoucím odměnám.

Po učení získáme politiku

$$\pi(s) = \arg \max_{a \in \mathcal{A}(s)} Q(s, a).$$

Během trénování však nemůžeme vždy volit pouze akci s nejvyšší známou hodnotou. Počáteční odhady mohou být nepřesné a agent by některé akce vůbec nevyzkoušel. Jednoduchou strategií je ε -hladová volba: s pravděpodobností $1 - \varepsilon$ použijeme dosud nejlepší akci a s pravděpodobností ε vybereme náhodnou přípustnou akci.

Algoritmus 11.1: Q-LEARNING

Vstup: Množiny stavů a akcí, rychlost učení α , diskont γ , pravděpodobnost průzkumu ε a počet epizod T

pro každou přípustnou dvojici (s, a) **opakuj**

$Q(s, a) \leftarrow 0$

pro $e \leftarrow 1, 2, \dots, T$ **opakuj**

$s \leftarrow$ počáteční stav

dokud stav s není koncový **opakuj**

s s pravděpodobností ε zvol náhodnou akci $a \in \mathcal{A}(s)$, jinak $a \leftarrow \arg \max_b Q(s, b)$

proved a a pozoruj odměnu r a nový stav s'

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_b Q(s', b) - Q(s, a)]$

$s \leftarrow s'$

Výstup: Politika $\pi(s) = \arg \max_{a \in \mathcal{A}(s)} Q(s, a)$

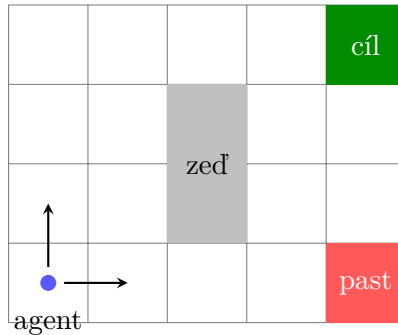


Figure 14: Jednoduché prostředí pro zpětnovazební učení.

Program 11.1 implementuje Q-learning v malém mřížkovém světě. Agent dostává za běžný krok odměnu -1 , za dosažení cíle 10 a za vstup do pasti -10 . S pravděpodobností ε zvolí náhodnou akci, a tím prostředí prozkoumává.

Jedna cesta od počátečního do koncového stavu tvoří *epizodu*. Záporná odměna za každý běžný krok vede agenta k tomu, aby nehledal pouze bezpečnou, ale také krátkou cestu. Kdyby běžný krok neměl žádnou cenu, všechny bezpečné cesty do cíle by mohly mít stejný celkový zisk bez ohledu na svou délku.

Program 11.1: Q-learning v jednoduchém mřížkovém světě.

```
1 import random
2
3
4 WIDTH, HEIGHT = 4, 3
5 START = (0, 0)
6 GOAL = (3, 2)
7 TRAP = (3, 0)
8 WALL = (1, 1)
9 ACTIONS = ("up", "right", "down", "left")
10 MOVE = {
11     "up": (0, 1),
12     "right": (1, 0),
13     "down": (0, -1),
14     "left": (-1, 0),
15 }
16
17
18 def step(state, action):
19     dx, dy = MOVE[action]
20     candidate = (state[0] + dx, state[1] + dy)
21     x, y = candidate
22     if not (0 <= x < WIDTH and 0 <= y < HEIGHT) or candidate == WALL:
23         candidate = state
24     if candidate == GOAL:
25         return candidate, 10, True
26     if candidate == TRAP:
27         return candidate, -10, True
28     return candidate, -1, False
29
30
31 def train(epochs=3000, alpha=0.2, gamma=0.9, epsilon=0.1):
32     states = [
33         (x, y)
34         for x in range(WIDTH)
35         for y in range(HEIGHT)
36         if (x, y) != WALL
37     ]
38     q = {(state, action): 0.0 for state in states for action in ACTIONS}
39
40     for _ in range(epochs):
41         state = START
42         done = False
43         while not done:
44             if random.random() < epsilon:
45                 action = random.choice(ACTIONS)
46             else:
47                 action = max(ACTIONS, key=lambda a: q[state, a])
```

```

49         new_state, reward, done = step(state, action)
50         continuation = 0.0 if done else max(
51             q[new_state, a] for a in ACTIONS
52         )
53         q[state, action] += alpha * (
54             reward + gamma * continuation - q[state, action]
55         )
56         state = new_state
57
58     return q
59
60
61 if __name__ == "__main__":
62     q = train()
63     policy = {
64         state: max(ACTIONS, key=lambda action: q[state, action])
65         for state in {state for state, _ in q}
66         if state not in (GOAL, TRAP)
67     }
68     print(policy)

```

Q-learning nepotřebuje předem znát pravidla přechodu prostředí. Hodnoty Q upravuje pouze podle skutečně získaných zkušeností. Pro velké nebo spojitě množiny stavů však tabulka nestačí a hodnotu $Q(s, a)$ je nutné aproximovat modelem.

Po skončení učení už průzkum nepotřebujeme a pro každý stav volíme akci s nejvyšší hodnotou Q . Naučená tabulka však platí pouze pro prostředí a odměny, na nichž vznikla. Změní-li se například poloha překážky nebo cena kroku, musí agent své odhady znovu upravit.

12 Lineární regrese

Lineární regrese hledá funkci

$$f(x) = ax + b,$$

která co nejlépe popisuje vztah mezi statistickými soubory

$$\mathbf{X} = (x_1, \dots, x_n), \quad \mathbf{Y} = (y_1, \dots, y_n).$$

Hodnota

$$\hat{y}_i = f(x_i)$$

je předpověď modelu pro vstup x_i , zatímco

$$e_i = y_i - \hat{y}_i$$

nazýváme *reziduum*. Kladné reziduum znamená, že bod leží nad regresní přímkou, záporné reziduum že leží pod ní.

Definice 12.1. Necht $f : \mathbb{R} \rightarrow \mathbb{R}$. Součet čtvercových chyb a střední kvadratickou chybu funkce f definujeme jako

$$\text{SSE}_{\mathbf{X}, \mathbf{Y}}(f) = \sum_{i=1}^n (y_i - f(x_i))^2$$

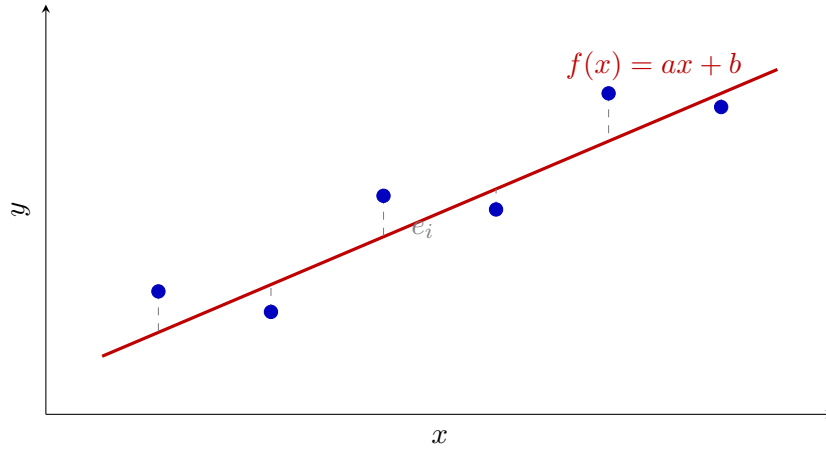


Figure 15: Předpovědi regresní přímky a svislá rezidua.

a

$$\text{MSE}_{\mathbf{X}, \mathbf{Y}}(f) = \frac{1}{n} \text{SSE}_{\mathbf{X}, \mathbf{Y}}(f).$$

Pouhý součet reziduí není vhodný, protože kladné a záporné chyby se mohou vyrušit. Čtverce jsou vždy nezáporné a větší chyby navíc zvýrazní. Minimalizace SSE a MSE vede ke stejné přímce, protože se obě veličiny liší pouze kladným násobkem $1/n$.

12.1 Metoda nejmenších čtverců

Pro model $f_{a,b}(x) = ax + b$ hledáme

$$(a, b) = \arg \min_{(a,b) \in \mathbb{R}^2} \sum_{i=1}^n (y_i - ax_i - b)^2.$$

Nejprve zvolme pevné a . Výraz lze chápat jako součet čtvercových odchylek hodnot statistického souboru

$$\mathbf{Y} - a\mathbf{X} = (y_1 - ax_1, \dots, y_n - ax_n)$$

od konstanty b . Podle minimalizační vlastnosti průměru je nejlepší volbou

$$b = \overline{\mathbf{Y} - a\mathbf{X}} = \bar{\mathbf{Y}} - a\bar{\mathbf{X}}.$$

Každý zbývajících kandidát proto prochází bodem

$$(\bar{\mathbf{X}}, \bar{\mathbf{Y}})$$

a má tvar

$$f(x) = \bar{\mathbf{Y}} + a(x - \bar{\mathbf{X}}).$$

Tvrzení 12.2. Nechť $\text{var}(\mathbf{X}) > 0$. Potom existuje právě jedna lineární funkce $f(x) = ax + b$, která minimalizuje $\text{SSE}_{\mathbf{X}, \mathbf{Y}}$, a její koeficienty jsou

$$a = \frac{\text{cov}(\mathbf{X}, \mathbf{Y})}{\text{var}(\mathbf{X})}, \quad b = \bar{\mathbf{Y}} - a\bar{\mathbf{X}}.$$

Proof. Z předchozí úvahy stačí minimalizovat chybu přímek procházejících bodem průměrů. Položme

$$u_i = x_i - \bar{\mathbf{X}}, \quad v_i = y_i - \bar{\mathbf{Y}}.$$

Po dosazení $b = \bar{\mathbf{Y}} - a\bar{\mathbf{X}}$ dostaneme

$$\begin{aligned} \text{SSE}_{\mathbf{X}, \mathbf{Y}}(f) &= \sum_{i=1}^n (v_i - au_i)^2 \\ &= \sum_{i=1}^n v_i^2 - 2a \sum_{i=1}^n u_i v_i + a^2 \sum_{i=1}^n u_i^2. \end{aligned}$$

Označme

$$a^* = \frac{\sum_{i=1}^n u_i v_i}{\sum_{i=1}^n u_i^2}.$$

Jmenovatel je kladný, protože $\text{var}(\mathbf{X}) > 0$. Doplněním na čtverec získáme

$$\begin{aligned} \text{SSE}_{\mathbf{X}, \mathbf{Y}}(f) &= \sum_{i=1}^n v_i^2 - \frac{(\sum_{i=1}^n u_i v_i)^2}{\sum_{i=1}^n u_i^2} \\ &\quad + \left(\sum_{i=1}^n u_i^2 \right) (a - a^*)^2. \end{aligned}$$

Poslední člen je nezáporný a je nulový právě pro $a = a^*$. Minimum je proto jediné a

$$a^* = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{\mathbf{X}})(y_i - \bar{\mathbf{Y}})}{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{\mathbf{X}})^2} = \frac{\text{cov}(\mathbf{X}, \mathbf{Y})}{\text{var}(\mathbf{X})}.$$

Jednoznačnost b pak plyne ze vztahu $b = \bar{\mathbf{Y}} - a\bar{\mathbf{X}}$. □

Pomocí korelace lze směrnici zapsat také

$$a = \rho_{\mathbf{XY}} \frac{\sigma_{\mathbf{Y}}}{\sigma_{\mathbf{X}}},$$

a regresní přímku ve tvaru

$$f(x) = \bar{\mathbf{Y}} + \rho_{\mathbf{XY}} \frac{\sigma_{\mathbf{Y}}}{\sigma_{\mathbf{X}}} (x - \bar{\mathbf{X}}).$$

Znaménko směrnice je tedy stejné jako znaménko korelace.

Uvedené vztahy poskytují přímo algoritmus pro trénování jednoduché lineární regrese. Není třeba zkoušet různé přímky ani opakovaně upravovat parametry: stačí jedním průchodem dat spočítat potřebné součty, z nich průměry, rozptyl a kovarianci a nakonec koeficienty a, b .

Algoritmus 12.1: LINEAR-REGRESSION

Vstup: Statistické soubory $\mathbf{X} = (x_1, \dots, x_n)$ a $\mathbf{Y} = (y_1, \dots, y_n)$ s $\text{var}(\mathbf{X}) > 0$

$$\bar{\mathbf{X}} \leftarrow \frac{1}{n} \sum_{i=1}^n x_i, \quad \bar{\mathbf{Y}} \leftarrow \frac{1}{n} \sum_{i=1}^n y_i$$

$$\text{var}(\mathbf{X}) \leftarrow \frac{1}{n} \sum_{i=1}^n (x_i - \bar{\mathbf{X}})^2$$

$$\text{cov}(\mathbf{X}, \mathbf{Y}) \leftarrow \frac{1}{n} \sum_{i=1}^n (x_i - \bar{\mathbf{X}})(y_i - \bar{\mathbf{Y}})$$

$$a \leftarrow \frac{\text{cov}(\mathbf{X}, \mathbf{Y})}{\text{var}(\mathbf{X})}$$

$$b \leftarrow \bar{\mathbf{Y}} - a\bar{\mathbf{X}}$$

Výstup: Regresní přímka $f(x) = ax + b$

12.2 Minimální MSE regresní přímky

Tvrzení 12.3. Necht $\sigma_{\mathbf{X}} > 0$, $\sigma_{\mathbf{Y}} > 0$ a f je regresní přímka souborů $\mathbf{X}$, $\mathbf{Y}$. Potom

$$\text{MSE}_{\mathbf{X},\mathbf{Y}}^{\min} = \min_{\ell \text{ lineární}} \text{MSE}_{\mathbf{X},\mathbf{Y}}(\ell) = \text{MSE}_{\mathbf{X},\mathbf{Y}}(f) = \sigma_{\mathbf{Y}}^2 (1 - \rho_{\mathbf{X}\mathbf{Y}}^2).$$

Proof. Použijeme označení u_i, v_i z předchozího důkazu. Pro optimální směrnicí

$$a = \frac{\sum_{i=1}^n u_i v_i}{\sum_{i=1}^n u_i^2}$$

platí

$$\begin{aligned} \text{SSE}_{\mathbf{X},\mathbf{Y}}(f) &= \sum_{i=1}^n (v_i - a u_i)^2 \\ &= \sum_{i=1}^n v_i^2 - 2a \sum_{i=1}^n u_i v_i + a^2 \sum_{i=1}^n u_i^2 \\ &= \sum_{i=1}^n v_i^2 - \frac{(\sum_{i=1}^n u_i v_i)^2}{\sum_{i=1}^n u_i^2}. \end{aligned}$$

Z definice korelace máme

$$\rho_{\mathbf{X}\mathbf{Y}}^2 = \frac{(\sum_{i=1}^n u_i v_i)^2}{(\sum_{i=1}^n u_i^2) (\sum_{i=1}^n v_i^2)}.$$

Druhý člen předchozího výrazu je proto

$$\frac{(\sum_{i=1}^n u_i v_i)^2}{\sum_{i=1}^n u_i^2} = \rho_{\mathbf{X}\mathbf{Y}}^2 \sum_{i=1}^n v_i^2.$$

Dostáváme

$$\text{SSE}_{\mathbf{X},\mathbf{Y}}(f) = (1 - \rho_{\mathbf{X}\mathbf{Y}}^2) \sum_{i=1}^n v_i^2.$$

Po vydělení n a použití

$$\frac{1}{n} \sum_{i=1}^n v_i^2 = \text{var}(\mathbf{Y}) = \sigma_{\mathbf{Y}}^2$$

získáme požadovaný vztah. □

12.3 Implementace v Pythonu

Program 12.1 nejprve spočítá průměry, rozptyl a kovarianci a potom přímo použije vzorce z tvrzení o regresní přímce.

Metoda `fit` kontroluje také předpoklad $\text{var}(\mathbf{X}) > 0$. Kdyby byly všechny vstupy stejné, data by ležela na jediné svislé přímce a směrnicí funkce $y = ax + b$ by z nich nebylo možné jednoznačně určit. Metoda proto místo dělení nulou oznámí chybu.

Program 12.1: Jednoduchá lineární regrese metodou nejmenších čtverců.

```
1 class LinearRegression:
2     def __init__(self):
3         self.a = 0.0
```

```

4     self.b = 0.0
5
6     def fit(self, x, y):
7         if len(x) != len(y) or not x:
8             raise ValueError("x and y must have the same non-zero length")
9
10        mean_x = sum(x) / len(x)
11        mean_y = sum(y) / len(y)
12        variance_x = sum((value - mean_x) ** 2 for value in x) / len(x)
13        if variance_x == 0:
14            raise ValueError("all x values are equal")
15
16        covariance = sum(
17            (xi - mean_x) * (yi - mean_y)
18            for xi, yi in zip(x, y)
19        ) / len(x)
20        self.a = covariance / variance_x
21        self.b = mean_y - self.a * mean_x
22        return self
23
24    def predict(self, x):
25        return [self.a * value + self.b for value in x]
26
27    def mse(self, x, y):
28        predictions = self.predict(x)
29        return sum(
30            (yi - prediction) ** 2
31            for yi, prediction in zip(y, predictions)
32        ) / len(y)
33
34
35 if __name__ == "__main__":
36     x = [1, 2, 3, 4, 5]
37     y = [2, 3, 5, 4, 6]
38     model = LinearRegression().fit(x, y)
39     print(f"f(x) = {model.a:.2f}x + {model.b:.2f}")
40     print(f"MSE = {model.mse(x, y):.2f}")

```

Trénování vyžaduje pouze konečný počet průchodů seznamy x a y , takže má časovou složitost $\mathcal{O}(n)$ a kromě vstupních dat spotřebuje konstantní pomocnou paměť. Po natrénování je jedna předpověď jen výpočtem $ax + b$, tedy operací v konstantním čase.

13 Koeficient determinace

Samotná hodnota MSE závisí na měřítku výstupního souboru. Abychom chybu regresní přímky zasadili do kontextu, porovnáme ji s nejlepším konstantním modelem

$$c(x) = \overline{Y}.$$

Podle vlastností průměru platí

$$\text{MSE}_{\mathbf{X}, \mathbf{Y}}(c) = \frac{1}{n} \sum_{i=1}^n (y_i - \overline{Y})^2 = \text{var}(\mathbf{Y}).$$

Definice 13.1. Necht $\sigma_Y > 0$ a f je regresní přímka souborů $\mathbf{X}, \mathbf{Y}$. *Koeficient determinace* definujeme jako

$$R^2 = 1 - \frac{\text{MSE}_{\mathbf{X}, \mathbf{Y}}(f)}{\text{MSE}_{\mathbf{X}, \mathbf{Y}}(x \mapsto \bar{\mathbf{Y}})} = 1 - \frac{\text{MSE}_{\mathbf{X}, \mathbf{Y}}(f)}{\text{var}(\mathbf{Y})}.$$

Pro jednoduchou lineární regresi dosadíme vztah z tvrzení o minimální MSE:

$$R^2 = 1 - \frac{\sigma_Y^2(1 - \rho_{\mathbf{X}\mathbf{Y}}^2)}{\sigma_Y^2} = \rho_{\mathbf{X}\mathbf{Y}}^2.$$

Proto $0 \leq R^2 \leq 1$. Hodnota blízká jedné znamená, že regresní přímka odstranila velkou část čtvercové chyby konstantního modelu. Hodnota blízká nule znamená, že lineární závislost přinesla jen malé zlepšení.

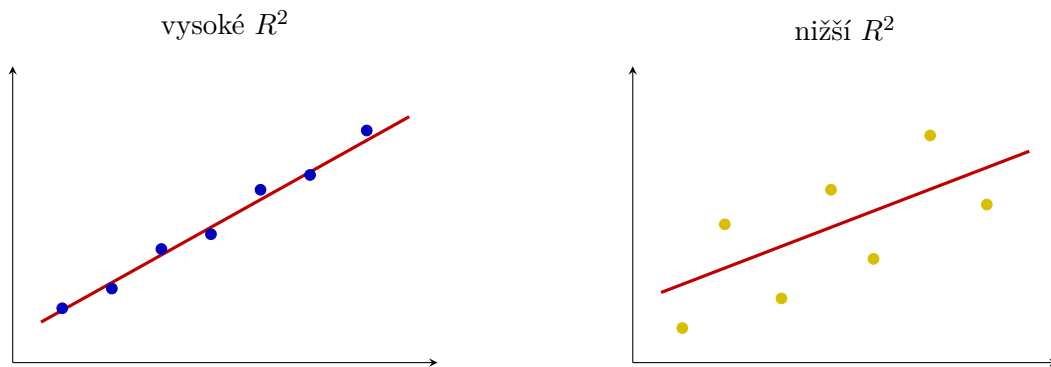


Figure 16: Stejný trend může být daty zachycen s různou přesností.

Příklad 13.2. Pro

$$\mathbf{X} = (1, 2, 3, 4, 5), \quad \mathbf{Y} = (2, 3, 5, 4, 6)$$

dostaneme

$$f(x) = 0,9x + 1,3, \quad \text{MSE}_{\mathbf{X}, \mathbf{Y}}(f) = 0,38.$$

Konstantní model má chybu

$$\text{var}(\mathbf{Y}) = 2,$$

a tedy

$$R^2 = 1 - \frac{0,38}{2} = 0,81.$$

Regresní přímka odstranila 81 % střední kvadratické chyby nejlepšího konstantního modelu.

Vysoké R^2 nedokazuje příčinný vztah ani správnost modelu mimo rozsah dat. V obecnějších regresních úlohách se navíc vztah $R^2 = \rho_{\mathbf{X}\mathbf{Y}}^2$ nemusí použít v této jednoduché podobě.

14 Omezení lineární regrese

Lineární regrese je jednoduchá, názorná a dobře interpretovatelná. Právě její jednoduchost je však zároveň omezením. Před použitím je vhodné prohlédnout bodový graf i rezidua a ověřit, zda lineární popis odpovídá povaze dat.

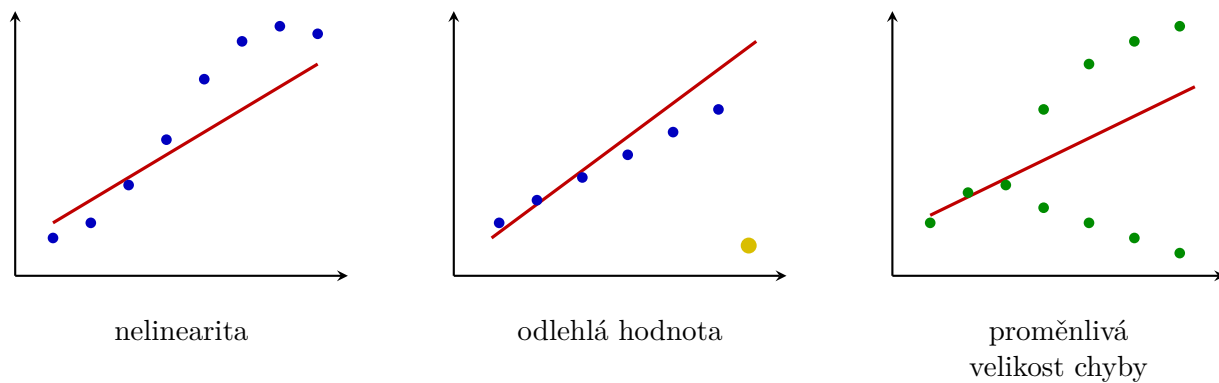


Figure 17: Tři typické problémy lineárního modelu.

- **Nelineární vztah.** Body mohou sledovat křivku, kterou žádná přímka nevystihne. Rezidua potom nevypadají náhodně, ale vytvářejí systematický obrazec.
- **Odlehlé hodnoty.** Metoda nejmenších čtverců zvýrazňuje velká rezidua. Jediný vzdálený bod proto může znatelně změnit směrnici i absolutní člen.
- **Proměnlivá velikost chyby.** Rozptyl reziduí může s rostoucím vstupem růst. Jediná hodnota MSE pak zakrývá, že model je v různých částech rozsahu různě přesný.
- **Závislá pozorování.** Hodnoty měřené v čase nebo opakovaně u stejného objektu mohou být vzájemně závislé. Běžná interpretace výsledků lineární regrese s tím nemusí počítat.
- **Extrapolace.** Přímka je určena daty v pozorovaném rozsahu. Daleko za tímto rozsahem se může skutečný vztah chovat zcela jinak.
- **Korelace není kauzalita.** Regresní přímka popisuje statistický vztah, ale sama neprokazuje, že změna $\mathbf{X}$ způsobuje změnu $\mathbf{Y}$.

Model není správný jen proto, že jej lze vypočítat. Výsledek je nutné posuzovat společně s grafem dat, rezidui, způsobem sběru dat a otázkou, pro kterou má být použit.

15 Úvod do klasifikace

Klasifikace je úloha učení s učitelem, ve které výstup není libovolné reálné číslo, ale jedna z konečně mnoha tříd. Množinu tříd označíme

$$\mathcal{Y} = \{c_1, \dots, c_K\}.$$

Vstupní data budeme popisovat statistickými soubory

$$\mathbf{X}_1, \dots, \mathbf{X}_p,$$

výstupní třídy souborem

$$\mathbf{Y} = (y_1, \dots, y_n), \quad y_i \in \mathcal{Y},$$

a i -tý objekt vektorem

$$\mathbf{x}_i = (x_{i1}, \dots, x_{ip}) \in \mathbb{R}^p.$$

Definice 15.1. *Klasifikátor* je funkce

$$h : \mathbb{R}^p \rightarrow \mathcal{Y},$$

která každému vektoru znaků $\mathbf{x} \in \mathbb{R}^p$ přiřadí jednu třídu $h(\mathbf{x}) \in \mathcal{Y}$.

Pro $\mathcal{Y} = \{0, 1\}$ hovoříme o binární klasifikaci. Je-li $K \geq 3$, jde o klasifikaci vícetřídní. U víceznačkové klasifikace může objekt nést několik značek současně; výstup pak lze zapsat vektorem z $\{0, 1\}^K$. Dále se zaměříme hlavně na binární klasifikaci.

15.1 Skóre, práh a rozhodovací hranice

Mnoho modelů nejprve vypočítá reálné skóre

$$s : \mathbb{R}^p \rightarrow \mathbb{R}$$

a teprve poté z něj určí třídu. Pro rozhodovací práh $t \in \mathbb{R}$ položíme

$$h^{(t)}(\mathbf{x}) = \begin{cases} 1, & s(\mathbf{x}) \geq t, \\ 0, & s(\mathbf{x}) < t. \end{cases}$$

Definice 15.2. *Rozhodovací hranice* skórovací funkce s při prahu t je množina

$$B_t = \{\mathbf{x} \in \mathbb{R}^p : s(\mathbf{x}) = t\}.$$

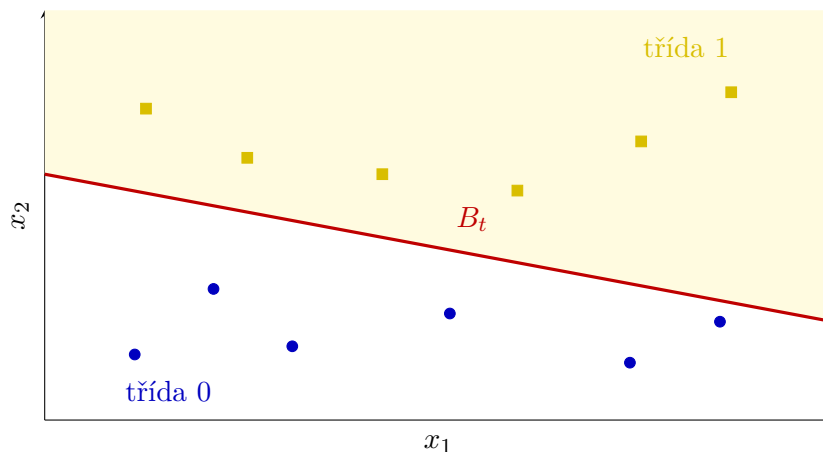


Figure 18: Rozhodovací hranice odděluje oblasti různých předpovědí.

Cílem není pouze správně klasifikovat trénovací objekty. Model má zachytit vztah mezi znaky a třídou tak, aby správně pracoval také s novými objekty. Příliš složitá hranice může obejít každý trénovací bod, ale na neznámých datech selhat.

16 Logistická regrese

Při binární klasifikaci používáme třídy 0 a 1. Přímé použití lineární regrese je problematické, protože funkce $ax + b$ může nabývat libovolných reálných hodnot a čtvercová chyba není přizpůsobena vyhodnocování jistoty klasifikační odpovědi. Logistická regrese proto lineární skóre převádí do intervalu $(0, 1)$.

Definice 16.1. *Logistická funkce* neboli *sigmoida* je funkce

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z \in \mathbb{R}.$$

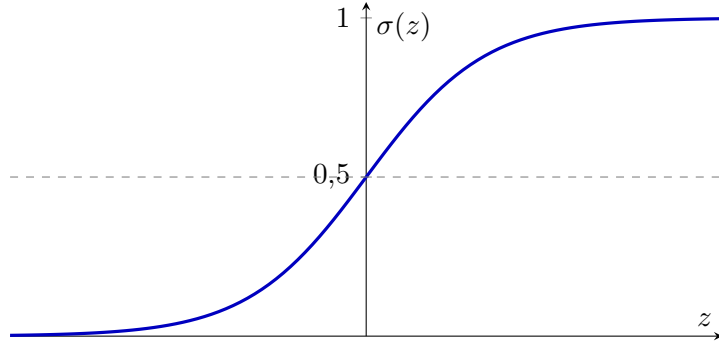


Figure 19: Logistická funkce převádí reálné skóre do intervalu $(0, 1)$.

Funkce je rostoucí, platí $\sigma(0) = 0,5$ a její hodnoty se pro velmi záporná, respektive velmi kladná z , blíží 0, respektive 1.

Pro jeden vstupní statistický soubor

$$\mathbf{X} = (x_1, \dots, x_n)$$

logistický model nejprve vypočítá lineární skóre $z_i = ax_i + b$ a potom

$$\hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-(ax_i + b)}}.$$

Statistický soubor výstupů označíme

$$\hat{\mathbf{P}} = (\hat{p}_1, \dots, \hat{p}_n).$$

Hodnotu $\hat{p}_i$ chápeme jako míru podpory modelu pro třídu 1.

Pro rozhodovací práh $t \in (0, 1)$ definujeme

$$h^{(t)}(x) = \begin{cases} 1, & \hat{p}(x) \geq t, \\ 0, & \hat{p}(x) < t. \end{cases}$$

Protože je sigmoida rostoucí,

$$\hat{p}(x) \geq t \iff ax + b \geq \ln\left(\frac{t}{1-t}\right).$$

Pro obvyklý práh $t = 0,5$ rozhoduje znaménko výrazu $ax + b$.

16.1 Křížová entropie

Definice 16.2. Pro skutečnou třídu $y_i \in \{0, 1\}$ a výstup $\hat{p}_i \in (0, 1)$ definujeme *binární křížovou entropii*

$$\ell(y_i, \hat{p}_i) = -(y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i)).$$

Průměrná křížová entropie na celém souboru je

$$L(\mathbf{Y}, \hat{\mathbf{P}}) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{p}_i).$$

Je-li $y_i = 1$, ztráta má tvar $-\ln \hat{p}_i$; je-li $y_i = 0$, má tvar $-\ln(1 - \hat{p}_i)$. Správná a jistá předpověď má malou ztrátu, zatímco jistá chybná předpověď je potrestána velmi vysokou hodnotou.

Parametry logistické regrese hledáme tak, aby

$$(a^*, b^*) = \arg \min_{(a,b) \in \mathbb{R}^2} L(a, b),$$

kde

$$L(a, b) = -\frac{1}{n} \sum_{i=1}^n (y_i \ln \hat{p}_i(a, b) + (1 - y_i) \ln(1 - \hat{p}_i(a, b))).$$

16.2 Odbočka: co vyjadřuje derivace

K hledání minima potřebujeme vědět, jak se hodnota funkce změní při malé změně jejího vstupu. Pro funkci jedné proměnné g porovnáváme hodnoty $g(c)$ a $g(c + h)$. Podíl

$$\frac{g(c + h) - g(c)}{h}$$

je směrnice sečny vedené dvěma blízkými body grafu. Necháme-li nenulové h přibližovat k nule a směrnice se ustálí, získáme *derivaci* $g'(c)$. Derivace tedy popisuje okamžitý směr a rychlost změny:

- $g'(c) > 0$: funkce v okolí c roste;
- $g'(c) < 0$: funkce v okolí c klesá;
- $g'(c) = 0$: graf má v daném bodě vodorovnou tečnu; může zde ležet minimum nebo maximum.

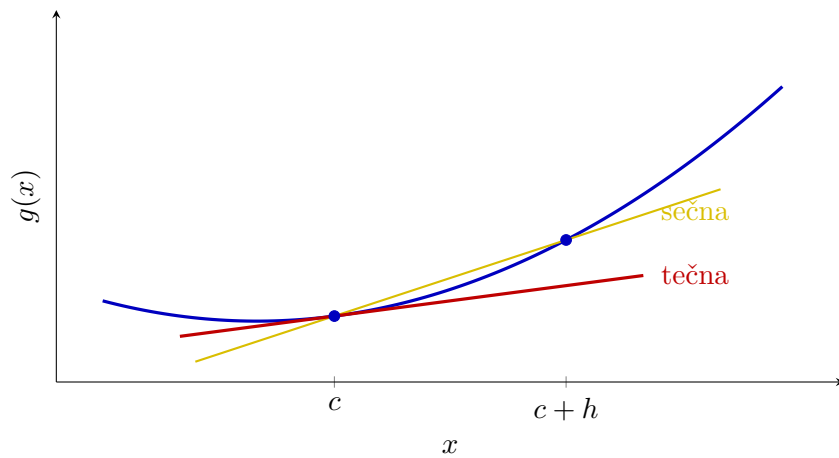


Figure 20: Sečna se při zmenšování h blíží tečně; její směrnice je derivací.

Závisí-li ztráta $L(a, b)$ na dvou parametrech, můžeme měnit vždy jen jeden z nich. Symboly

$$\frac{\partial L}{\partial a}, \quad \frac{\partial L}{\partial b}$$

označují derivace podle a , respektive podle b , při pevném druhém parametru. Vektor

$$\nabla L(a, b) = \left(\frac{\partial L}{\partial a}, \frac{\partial L}{\partial b} \right)$$

nazýváme *gradient*. Ukazuje směrem nejrychlejšího růstu ztráty, a proto se při minimalizaci pohybujeme opačným směrem.

16.3 Gradientní sestup

Pro logistickou regresi lze odvodit

$$\frac{\partial L}{\partial a} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i) x_i, \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i).$$

Při rychlosti učení $\eta > 0$ opakujeme aktualizace

$$a \leftarrow a - \eta \frac{\partial L}{\partial a}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}.$$

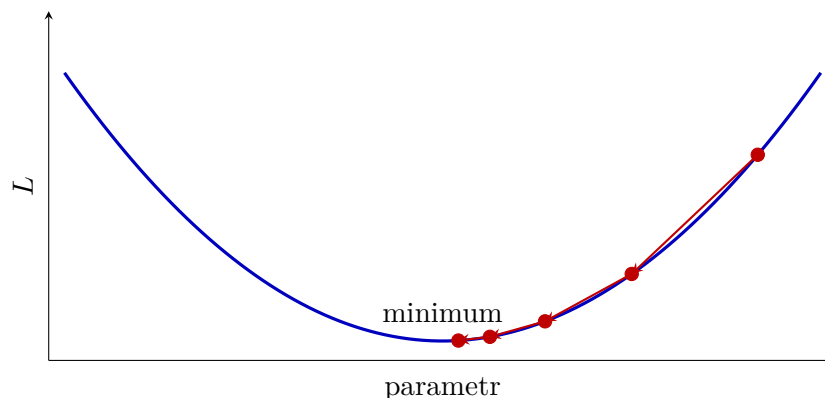


Figure 21: Gradientní sestup postupně snižuje hodnotu ztrátové funkce.

Příliš malá η vede k pomalému postupu. Příliš velká hodnota může minimum přeskokovat nebo výpočet rozkmitat. Zastavujeme například po pevném počtu iterací nebo ve chvíli, kdy se ztráta a parametry mění už jen nepatrně.

Jedna iterace využije všechny trénovací objekty k výpočtu průměrného gradientu. Takové variantě říkáme *dávkový gradientní sestup*. U velkých souborů lze gradient odhadovat z menších náhodných dávek; aktualizace jsou potom levnější, ale jejich směr více kolísá.

Algoritmus 16.1: LOGISTIC-REGRESSION

Vstup: Soubory $\mathbf{X} = (x_1, \dots, x_n)$, $\mathbf{Y} = (y_1, \dots, y_n)$, rychlost učení $\eta > 0$ a počet iterací T

$a \leftarrow 0$, $b \leftarrow 0$

pro $r \leftarrow 1, 2, \dots, T$ **opakuj**

pro $i \leftarrow 1, 2, \dots, n$ **opakuj**

$$\hat{p}_i \leftarrow \frac{1}{1 + e^{-(ax_i + b)}}$$

$$g_a \leftarrow \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i) x_i$$

$$g_b \leftarrow \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i)$$

$$a \leftarrow a - \eta g_a, \quad b \leftarrow b - \eta g_b$$

Výstup: Parametry a, b a skóre $\hat{p}(x) = \sigma(ax + b)$

16.4 Implementace v Pythonu

Program 16.1 používá přesně uvedené dva gradienty. Metoda `fit` opakovaně vypočítá všechny hodnoty $\hat{p}_i$ a upraví parametry a, b ; metoda `predict` převede výstupy na třídy pomocí zvoleného prahu.

Funkce `sigmoid` je zapsána ve dvou algebraicky ekvivalentních větvích. Pro záporné z používá výraz e^z , aby při velmi záporném vstupu nemusela počítat obrovskou hodnotu e^{-z} . Jde o technickou úpravu numerické stability, nikoli o změnu matematického modelu.

Program 16.1: Logistická regrese pro jeden vstupní statistický znak.

```

1 from math import exp
2
3
4 def sigmoid(z):
5     if z >= 0:
6         return 1.0 / (1.0 + exp(-z))
7     value = exp(z)
8     return value / (1.0 + value)
9
10
11 class LogisticRegression:
12     def __init__(self, learning_rate=0.1, iterations=2000):
13         self.learning_rate = learning_rate
14         self.iterations = iterations
15         self.a = 0.0
16         self.b = 0.0
17
18     def fit(self, x, y):
19         if len(x) != len(y) or not x:
20             raise ValueError("x and y must have the same non-zero length")
21
22         n = len(x)
23         for _ in range(self.iterations):
24             probabilities = [
25                 sigmoid(self.a * xi + self.b)
26                 for xi in x
27             ]
28             gradient_a = sum(
29                 (probability - yi) * xi
30                 for xi, yi, probability in zip(x, y, probabilities)
31             ) / n
32             gradient_b = sum(
33                 probability - yi
34                 for yi, probability in zip(y, probabilities)
35             ) / n
36             self.a -= self.learning_rate * gradient_a
37             self.b -= self.learning_rate * gradient_b
38         return self
39
40     def predict_proba(self, x):
41         return [sigmoid(self.a * xi + self.b) for xi in x]
42
43     def predict(self, x, threshold=0.5):
44         return [
45             int(probability >= threshold)
46             for probability in self.predict_proba(x)
47         ]
48
49
50 if __name__ == "__main__":
51     x = [-3, -2, -1, 1, 2, 3]
52     y = [0, 0, 0, 1, 1, 1]
53     model = LogisticRegression().fit(x, y)

```

```

54 print(model.predict_proba([-1.5, 0, 1.5]))
55 print(model.predict([-1.5, 0, 1.5]))

```

Ukázka používá jeden vstupní znak, aby byl vztah mezi parametry a rozhodovací hranicí dobře vidět. Pro více znaků nahradíme $ax + b$ výrazem $\sum_{j=1}^p w_j x_j + \theta$ a pro každou váhu w_j počítáme vlastní složku gradientu; princip pseudokódu zůstává stejný.

17 Vyhodnocení klasifikace

U binární klasifikace porovnáváme skutečnou třídu $y_i \in \{0, 1\}$ s předpovědí $\hat{y}_i^{(t)} = h^{(t)}(\mathbf{x}_i)$. Mohou nastat čtyři výsledky:

- správně pozitivní: $y_i = 1$ a $\hat{y}_i^{(t)} = 1$;
- správně negativní: $y_i = 0$ a $\hat{y}_i^{(t)} = 0$;
- falešně pozitivní: $y_i = 0$, ale $\hat{y}_i^{(t)} = 1$;
- falešně negativní: $y_i = 1$, ale $\hat{y}_i^{(t)} = 0$.

Jejich počty značíme

$$TP_{\mathbf{X}, \mathbf{Y}}^{(t)}, \quad TN_{\mathbf{X}, \mathbf{Y}}^{(t)}, \quad FP_{\mathbf{X}, \mathbf{Y}}^{(t)}, \quad FN_{\mathbf{X}, \mathbf{Y}}^{(t)}.$$

Například

$$TP_{\mathbf{X}, \mathbf{Y}}^{(t)} = \left| \left\{ i : y_i = 1, h^{(t)}(\mathbf{x}_i) = 1 \right\} \right|.$$

Ostatní tři počty definujeme analogicky. Je-li z kontextu jasný soubor i práh, budeme jejich dolní a horní indexy někdy vynechávat.

	předpověď 1	předpověď 0
skutečnost 1	TP správně pozitivní	FN falešně negativní
skutečnost 0	FP falešně pozitivní	TN správně negativní

Figure 22: Matice záměn pro binární klasifikaci.

17.1 Správnost, přesnost, pokrytí a F_1 -míra

Definice 17.1. Jsou-li příslušné jmenovatele nenulové, definujeme

$$\begin{aligned}\text{Acc}_{\mathbf{X},\mathbf{Y}}^{(t)} &= \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \\ \text{Prec}_{\mathbf{X},\mathbf{Y}}^{(t)} &= \frac{\text{TP}}{\text{TP} + \text{FP}}, \\ \text{Rec}_{\mathbf{X},\mathbf{Y}}^{(t)} &= \frac{\text{TP}}{\text{TP} + \text{FN}}, \\ F_{1,\mathbf{X},\mathbf{Y}}^{(t)} &= \frac{2}{(\text{Prec}_{\mathbf{X},\mathbf{Y}}^{(t)})^{-1} + (\text{Rec}_{\mathbf{X},\mathbf{Y}}^{(t)})^{-1}}.\end{aligned}$$

Správnost (anglicky *accuracy*) odpovídá podílu všech správných odpovědí. *Přesnost* (anglicky *precision*) se ptá, kolika pozitivním předpovědím můžeme věřit. *Pokrytí* (anglicky *recall*) se ptá, jakou část skutečně pozitivních objektů jsme našli. F_1 -míra je harmonický průměr přesnosti a pokrytí a je vysoká pouze tehdy, jsou-li vysoké obě hodnoty.

Dosazením definic dostaneme užitečný tvar

$$F_{1,\mathbf{X},\mathbf{Y}}^{(t)} = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}}.$$

Příklad 17.2. Pro

$$\text{TP} = 36, \quad \text{TN} = 44, \quad \text{FP} = 8, \quad \text{FN} = 12$$

vychází

$$\text{Acc} = 0,80, \quad \text{Prec} \approx ,82, \quad \text{Rec} = 0,75, \quad F_1 \approx ,78.$$

Správnost může být zavádějící u nevyvážených tříd. Jestliže je mezi tisícem zpráv pouze deset podvodných, klasifikátor označující vždy běžnou zprávu dosáhne správnosti 0,99, ale jeho pokrytí podvodných zpráv je nulové.

17.2 Vliv prahu a neznámá data

Snížení rozhodovacího prahu obvykle zvýší počet pozitivních předpovědí. Tím může růst pokrytí, ale zároveň přibývají falešně pozitivní výsledky a klesá přesnost. Vhodný práh proto volíme podle důsledků obou typů chyb a podle výsledků na validačních datech.

V lékařském předběžném vyšetření může být závažnější přehlédnout nemocného člověka než poslat zdravého na další kontrolu; dává proto smysl upřednostnit vysoké pokrytí. U automatického blokování účtů může být naopak falešně pozitivní rozhodnutí velmi nákladné, a proto požadujeme vysokou přesnost. Jediná metrika bez kontextu tyto rozdíly nezachytí.

Změnou prahu získáme celou rodinu klasifikátorů ze stejného skórovacího modelu. Při jejich porovnávání sledujeme, jak se současně mění podíl správně pozitivních a falešně pozitivních odpovědí. Tím lze hodnotit kvalitu samotného pořadí objektů podle skóre odděleně od volby jednoho konkrétního prahu.

Metriky uváděné po dokončení vývoje modelu počítáme na testovacích datech, která nebyla použita ani k trénování, ani k volbě prahu či jiných nastavení. Jinak získáme příliš optimistický odhad kvality.

18 Algoritmus k -NN

Metoda k nejbližších sousedů (anglicky *k-nearest neighbors*, k -NN) vychází z intuice, že podobné objekty mívají stejnou třídu. Nevytváří předem parametrický vzorec. Při klasifikaci nového objektu přímo vyhledá nejbližší trénovací objekty a nechá je hlasovat.

Definice 18.1. Pro vektory

$$\mathbf{u} = (u_1, \dots, u_p), \quad \mathbf{v} = (v_1, \dots, v_p)$$

definujeme *eukleidovskou vzdálenost*

$$d_E(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{j=1}^p (u_j - v_j)^2}.$$

Obecně budeme vzdálenost značit $d(\mathbf{u}, \mathbf{v})$. Seřadíme indexy trénovacích objektů tak, aby

$$d(\mathbf{x}, \mathbf{x}_{\pi(1)}) \leq \dots \leq d(\mathbf{x}, \mathbf{x}_{\pi(n)}).$$

Množina indexů k nejbližších sousedů je

$$N_k(\mathbf{x}) = \{\pi(1), \dots, \pi(k)\}.$$

Definice 18.2. Klasifikátor k -NN je funkce

$$h^{(k)}(\mathbf{x}) = \arg \max_{c \in \mathcal{Y}} |\{i \in N_k(\mathbf{x}) : y_i = c\}|.$$

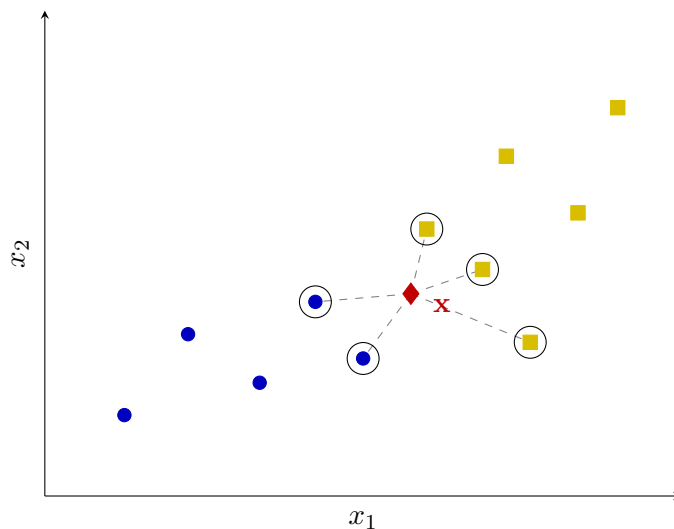


Figure 23: Klasifikace nového objektu podle pěti nejbližších sousedů.

Při shodě hlasů potřebujeme předem stanovené pravidlo. U binární klasifikace se proto často volí liché k . Malé k vytváří pružnou hranici citlivou na jednotlivé body a šum. Velké k rozhodování vyhlazuje, ale může přehlédnout malé místní skupiny. Hodnotu k proto volíme podle výsledku na validační množině.

Vedle eukleidovské vzdálenosti lze použít například Minkowského vzdálenost

$$d_q(\mathbf{u}, \mathbf{v}) = \left(\sum_{j=1}^p |u_j - v_j|^q \right)^{1/q}, \quad q \geq 1.$$

Pro $q = 1$ jde o manhattanskou vzdálenost, pro $q = 2$ o eukleidovskou. Smysl výsledku vždy závisí na tom, zda zvolená vzdálenost skutečně vystihuje podobnost objektů dané úlohy.

Před použitím k -NN je obvykle nutné standardizovat nebo normalizovat jednotlivé vstupní znaky. Jinak může znak s větším číselným měřítkem zcela ovládnout výpočet vzdálenosti.

Při použití klasifikátoru se tedy nejprve všechny vzdálenosti vypočítají, potom se indexy seřadí a nakonec se sečtou hlasy prvních k objektů. Pseudokód výslovně uvádí i jednoznačné pravidlo pro shodu hlasů; bez něj by algoritmus nebyl úplně určen.

Algoritmus 18.1: k -NN

Vstup: Trénovací množina $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, nový objekt $\mathbf{x}$, počet sousedů k a vzdálenost d
pro $i \leftarrow 1, 2, \dots, n$ **opakuji**
 $r_i \leftarrow d(\mathbf{x}, \mathbf{x}_i)$

seřadím indexy $\pi(1), \dots, \pi(n)$ **tak, aby** $r_{\pi(1)} \leq \dots \leq r_{\pi(n)}$

pro každý třída $c \in \mathcal{Y}$ **opakuji**

$H(c) \leftarrow \left| \left\{ j \in \{1, \dots, k\} : y_{\pi(j)} = c \right\} \right|$

Výstup: Třída s nejvyšší hodnotou $H(c)$; při shodě předem určená menší třída

18.1 Implementace v Pythonu

Program 18.1 uchovává trénovací objekty a jejich třídy. Při predikci spočítá všechny vzdálenosti, vybere k nejmenších a vrátí nejčastější třídu. V případě shody rozhodne menší hodnota třídy, aby byl výsledek jednoznačný.

Funkce `dist` z modulu `math` počítá eukleidovskou vzdálenost vektorů libovolné stejné délky. Třída `Counter` potom vytváří tabulku počtů hlasů. Samotné volání `fit` data pouze uloží, protože k -NN během trénovací fáze žádné parametry nehledá.

Program 18.1: Klasifikátor k -NN s eukleidovskou vzdáleností.

```
1 from collections import Counter
2 from math import dist
3
4
5 class KNNClassifier:
6     def __init__(self, k=3):
7         if k < 1:
8             raise ValueError("k must be positive")
9         self.k = k
10        self.x_train = []
11        self.y_train = []
12
13    def fit(self, x, y):
14        if len(x) != len(y) or not x:
15            raise ValueError("x and y must have the same non-zero length")
16        if self.k > len(x):
17            raise ValueError("k cannot exceed the number of objects")
18        self.x_train = list(x)
```

```

19     self.y_train = list(y)
20     return self
21
22     def predict_one(self, x):
23         neighbours = sorted(
24             zip(self.x_train, self.y_train),
25             key=lambda item: dist(x, item[0]),
26         )[:self.k]
27         votes = Counter(label for _, label in neighbours)
28         return min(votes, key=lambda label: (-votes[label], label))
29
30     def predict(self, x):
31         return [self.predict_one(point) for point in x]
32
33
34 if __name__ == "__main__":
35     x = [(1, 1), (2, 1), (2, 2), (6, 5), (7, 5), (7, 6)]
36     y = [0, 0, 0, 1, 1, 1]
37     model = KNNClassifier(k=3).fit(x, y)
38     print(model.predict([(2, 3), (6, 4)]))

```

Trénování je téměř okamžité, ale klasifikace nového objektu vyžaduje porovnání s celou trénovací množinou. Metoda proto přesouvá většinu výpočetní práce do okamžiku použití modelu.

Naivní predikce spočítá n vzdáleností a seřadí je, takže pro jeden objekt potřebuje čas $\mathcal{O}(np + n \log n)$. Pro velká data lze hledání urychlit prostorovými datovými strukturami nebo přibližným vyhledáváním sousedů, jejich účinnost však závisí na počtu znaků a geometrii dat.

19 Algoritmus SVM

Metoda podpůrných vektorů (anglicky *support vector machine*, SVM) hledá rozhodovací hranici, která nejen oddělí třídy, ale ponechá mezi nimi co nejširší volný prostor. Nejprve budeme uvažovat lineárně oddělitelná data.

19.1 Oddělující nadrovina a maximální okraj

Pro nový vektor parametrů

$$\mathbf{w} = (w_1, \dots, w_p) \in \mathbb{R}^p$$

a $\theta \in \mathbb{R}$ zavedeme lineární skóre

$$g(\mathbf{x}) = \sum_{j=1}^p w_j x_j + \theta.$$

Rozhodovací hranici tvoří nadrovina

$$g(\mathbf{x}) = 0.$$

V rovině jde o přímku, ve třech rozměrech o rovinu. Klasifikátor přiřadí třídu 1 bodům s $g(\mathbf{x}) \geq 0$ a třídu 0 bodům s $g(\mathbf{x}) < 0$.

Vzdálenost bodu $\mathbf{x}$ od této nadroviny je

$$\frac{|g(\mathbf{x})|}{\|\mathbf{w}\|}, \quad \|\mathbf{w}\| = \sqrt{\sum_{j=1}^p w_j^2}.$$

Samotných oddělovacích nadrovin může existovat mnoho. SVM volí takovou, jejíž nejbližší body z obou tříd jsou od hranice co nejdále. Tento nejmenší odstup nazýváme *okraj* (anglicky *margin*).

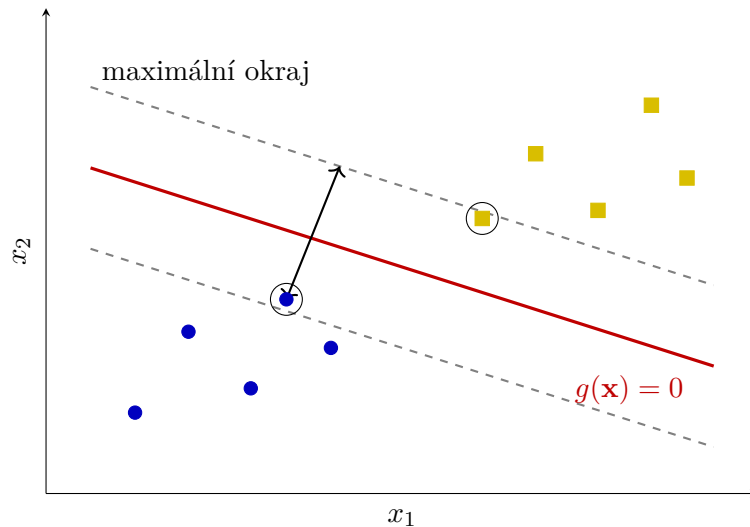


Figure 24: Oddělovací přímka, maximální okraj a podpůrné vektory.

Po vhodném přenásobení $\mathbf{w}$ a θ můžeme hraniční přímky okraje zapsat

$$g(\mathbf{x}) = 1 \quad \text{a} \quad g(\mathbf{x}) = -1.$$

Jejich vzájemná vzdálenost je

$$\frac{2}{\|\mathbf{w}\|}.$$

Body ležící na hranicích okraje nazýváme *podpůrné vektory*. Právě ony polohu optimální hranice přímo určují. Posun bodu ležícího daleko od okraje ji obvykle nezmění, zatímco posun podpůrného vektoru ano.

Šířku okraje nemůžeme zvětšovat libovolným přenásobením parametrů, protože současně požadujeme, aby trénovací objekty ležely na správné straně hraničních nadrovin. Označíme-li třídy hodnotami $\tilde{y}_i = 2y_i - 1 \in \{-1, 1\}$, tvrdý okraj splňuje podmínky

$$\tilde{y}_i g(\mathbf{x}_i) \geq 1 \quad \text{pro všechna } i.$$

Maximalizace šířky $2/\|\mathbf{w}\|$ je za těchto podmínek ekvivalentní minimalizaci $\frac{1}{2}\|\mathbf{w}\|^2$.

19.2 Neoddělitelná data a měkký okraj

Reálná data často nelze bez chyby oddělit jedinou nadrovinou. SVM proto může použít *měkký okraj*: připustí některé body uvnitř okraje nebo dokonce na chybné straně hranice. Nastavení modelu pak představuje kompromis mezi dvěma cíli:

- mít okraj co nejširší;
- mít co nejméně závažných porušení a chybných klasifikací.

Parametr $C > 0$ určuje cenu porušení. Velké C chyby silně trestá a obvykle vede k užšímu okraji. Menší C dovolí více porušení, ale může vytvořit stabilnější širší hranici. Hodnotu C volíme pomocí validačních dat.

Míru porušení pro objekt i lze vyjádřit *pantovou ztrátou* (anglicky *hinge loss*)

$$\ell_i = \max \{0, 1 - \tilde{y}_i g(\mathbf{x}_i)\}.$$

Je-li objekt správně za okrajem, platí $\ell_i = 0$. Bod uvnitř okraje má kladnou ztrátu, a bod na chybné straně rozhodovací hranice ztrátu větší než 1. Trénování měkkého okraje lze proto zapsat jako minimalizaci

$$\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \ell_i.$$

Lineární hranice nemusí stačit ani po povolení chyb. Jednou možností je převést objekty do prostoru s dalšími znaky, v němž je lze oddělit nadrovinou. Jádrový trik (anglicky *kernel trick*) umožňuje některé takové výpočty provést bez explicitního vytváření všech nových souřadnic. Výsledná hranice v původním prostoru pak může být nelineární.

Jádro musí vhodným způsobem vyjadřovat podobnost dvojice objektů. Volba lineárního jádra ponechá původní nadrovinu, polynomiální nebo gaussovské jádro umožní zakřivené hranice. Složitější hranice však nemusí lépe zobecňovat, a proto typ jádra i jeho parametry volíme podle validačních dat.

19.3 Vlastnosti a použití

Mezi výhody SVM patří jasná geometrická interpretace, možnost nelineárních hranic a skutečnost, že hranici přímo určuje jen část trénovacích objektů. Metoda může dobře fungovat i při větším počtu znaků.

Nevýhodou je citlivost na měřítko dat a na volbu C a jádra. U velkých datových souborů může být trénování náročné a výstup modelu nemá bez další úpravy přímou pravděpodobnostní interpretaci jako výstup logistické regrese. Typicky se SVM používá pro klasifikaci textu, obrazových znaků nebo biologických dat, zejména není-li počet trénovacích objektů extrémně velký.

Podpůrné vektory zároveň poskytují užitečnou kontrolu modelu. Pokud se jejich malá změna výrazně projeví na rozhodovací hranici, může být model citlivý na šum nebo odlehlá pozorování. Velký počet podpůrných vektorů zase naznačuje, že třídy nejsou zvolenými znaky dobře oddělené.

Stejně jako u k -NN je před trénováním SVM obvykle nutné standardizovat nebo normalizovat vstupní znaky. Jinak geometrický okraj odráží především rozdílné jednotky a měřítko.

Poznámka 19.1. Základy metody podpůrných vektorů vznikaly v pracích Vladimira Vapnika a Alexeje Červoněnkise už v 60. letech 20. století. Dnes běžnou variantu s měkkým okrajem popsali Corinna Cortesová a Vladimir Vapnik roku 1995.

19.4 Pseudokód a implementace

Pro názornou implementaci použijeme lineární SVM a stochastický podgradientní sestup. Místo parametru C budeme regulaci zapisovat pomocí nového parametru $\lambda > 0$ v ekvivalentně škálovaném cíli

$$\frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^n \max \{0, 1 - \tilde{y}_i g(\mathbf{x}_i)\}.$$

Větší λ silněji zmenšuje váhy a upřednostňuje širší okraj; jeho role je tedy opačná než role C .

V každém kroku nejprve váhy mírně zmenšíme kvůli členu $\frac{\lambda}{2} \|\mathbf{w}\|^2$. Pokud vybraný objekt porušuje okraj, přidáme navíc opravu ve směru $\tilde{y}_i \mathbf{x}_i$ a upravíme práh. Jde o zjednodušený výukový postup; plnohodnotné knihovny používají rychlejší optimalizační metody a pečlivější volbu rychlosti učení.

Algoritmus 19.1: SVM-SGD

Vstup: Trénovací množina D , rychlost učení $\eta > 0$, regulace $\lambda > 0$ a počet epoch T

$\mathbf{w} \leftarrow \mathbf{0}$, $\theta \leftarrow 0$

pro $e \leftarrow 1, 2, \dots, T$ **opakuj**

pro každý objekt $(\mathbf{x}_i, y_i) \in D$ **opakuj**

$\tilde{y}_i \leftarrow 2y_i - 1$

$\mathbf{w} \leftarrow (1 - \eta\lambda)\mathbf{w}$

pokud $\tilde{y}_i(\sum_{j=1}^p w_j x_{ij} + \theta) < 1$ **pak**

$\mathbf{w} \leftarrow \mathbf{w} + \eta \tilde{y}_i \mathbf{x}_i$

$\theta \leftarrow \theta + \eta \tilde{y}_i$

Výstup: Klasifikátor $h(\mathbf{x}) = 1$, pokud $\sum_{j=1}^p w_j x_j + \theta \geq 0$, jinak 0

Program 19.1 odpovídá tomuto pseudokódu. Metoda `decision_function` vrací podepsané skóre: jeho znaménko určuje třídu a jeho absolutní hodnota vyjadřuje vzdálenost od hranice pouze po vydělení normou vah. Metoda `predict` proto ke klasifikaci používá jen znaménko.

Program 19.1: Lineární SVM trénované stochastickým podgradientním sestupem.

```

1 def dot(u, v):
2     return sum(ui * vi for ui, vi in zip(u, v))
3
4
5 class LinearSVM:
6     def __init__(self, learning_rate=0.02, regularization=0.01,
7                 epochs=1000):
8         self.learning_rate = learning_rate
9         self.regularization = regularization
10        self.epochs = epochs
11        self.weights = []
12        self.theta = 0.0
13
14    def fit(self, x, y):
15        if len(x) != len(y) or not x:
16            raise ValueError("x and y must have the same non-zero length")
17        dimension = len(x[0])
18        if any(len(row) != dimension for row in x):
19            raise ValueError("all input vectors must have the same length")
20        if any(label not in (0, 1) for label in y):
21            raise ValueError("labels must be 0 or 1")
22
23        self.weights = [0.0] * dimension
24        self.theta = 0.0
25        for _ in range(self.epochs):
26            for vector, label in zip(x, y):
27                signed_label = 2 * label - 1
28                margin = signed_label * (
29                    dot(self.weights, vector) + self.theta
30                )
31
32                shrink = 1.0 - (
33                    self.learning_rate * self.regularization
34                )

```

```

35         self.weights = [shrink * weight for weight in self.weights]
36         if margin < 1.0:
37             self.weights = [
38                 weight + self.learning_rate * signed_label * value
39                 for weight, value in zip(self.weights, vector)
40             ]
41             self.theta += self.learning_rate * signed_label
42         return self
43
44     def decision_function(self, x):
45         return [dot(self.weights, vector) + self.theta for vector in x]
46
47     def predict(self, x):
48         return [int(score >= 0.0) for score in self.decision_function(x)]
49
50
51 if __name__ == "__main__":
52     x = [(-2, -1), (-1, -2), (-2, -2), (1, 2), (2, 1), (2, 2)]
53     y = [0, 0, 0, 1, 1, 1]
54     model = LinearSVM().fit(x, y)
55     print("weights:", [round(value, 3) for value in model.weights])
56     print("theta:", round(model.theta, 3))
57     print("predictions:", model.predict([(-1, 0), (0, 1), (3, 2)]))

```

Příklad obsahuje dvě lineárně oddělitelné skupiny bodů. Po natrénování program vypíše parametry hranice a třídy několika nových objektů. Před použitím na reálných datech bychom vstupní znaky standardizovali a hodnoty λ , η i počet epoch zvolili podle validační množiny.

20 Úvod do neuronových sítí

Lineární a logistická regrese skládají výstup z jediného lineárního skóre. Takový model vytváří lineární rozhodovací hranici a nemůže sám zachytit složitější vztahy. Neuronová síť spojuje větší počet jednoduchých výpočetních jednotek do vrstev. Výstup jedné vrstvy se stává vstupem vrstvy následující, takže síť může postupně vytvářet stále složitější reprezentace dat.

Název vznikl volnou inspirací biologickými neurony. Biologický neuron přijímá signály dendrity, zpracovává je v těle buňky a při dostatečném podráždění vysílá signál axonem. Formální neuron tento proces výrazně zjednodušuje: vstupy vynásobí vahami, sečte je, přidá práh a na výsledek použije aktivační funkci.

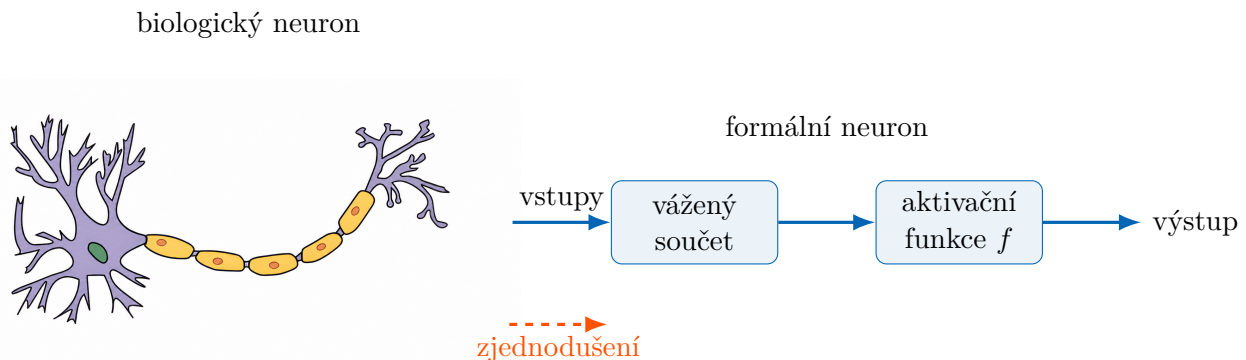


Figure 25: Biologická inspirace a její zjednodušení na formální neuron.

Neuronová síť není přesným modelem lidského mozku. Podobnost spočívá především v propojení mnoha jednoduchých jednotek. Způsob výpočtu i učení běžných sítí je matematickou konstrukcí navrženou pro zpracování dat.

Váha spoje určuje, jak silně daný vstup ovlivní výsledek. Kladná váha vliv podporuje, záporná jej potlačuje a váha blízka nule jej téměř ignoruje. Učení sítě spočívá v hledání takových vah a práhů, při nichž budou její výstupy na trénovacích datech co nejméně chybné.

Neuronové sítě se používají například pro rozpoznávání obrazu a řeči, zpracování textu, předpovídání časových řad nebo řízení. Jejich výhodou je schopnost modelovat složité nelineární vztahy. Cenou za tuto obecnost je větší množství dat a výpočtů potřebných k učení a také obtížnější vysvětlitelnost výsledku.

Poznámka 20.1. Frank Rosenblatt představil perceptron roku 1957 a první hardwarovou realizaci Mark I Perceptron dokončil roku 1958. Zpětné šíření chyby bylo známé v různých podobách dříve, ale jeho význam pro učení vícevrstevných sítí výrazně zpopularizovala práce Davida Rumelharta, Geoffreyho Hinton a Ronalda Williamse z roku 1986.

21 Formální neuronová síť

21.1 Formální neuron

Uvažujme objekt $\mathbf{x}_i = (x_{i1}, \dots, x_{ip}) \in \mathbb{R}^p$. Formální neuron má pro každý vstup váhu $w_j \in \mathbb{R}$, práh $\theta \in \mathbb{R}$ a aktivační funkci f . Nejprve vypočítá lineární kombinaci

$$z_i = \sum_{j=1}^p w_j x_{ij} + \theta$$

a potom výstup

$$\hat{y}_i = f(z_i).$$

Práh θ lze chápat jako váhu dodatečného stálého vstupu 1. V některých zdrojích se proto nazývá posunutí (anglicky *bias*).

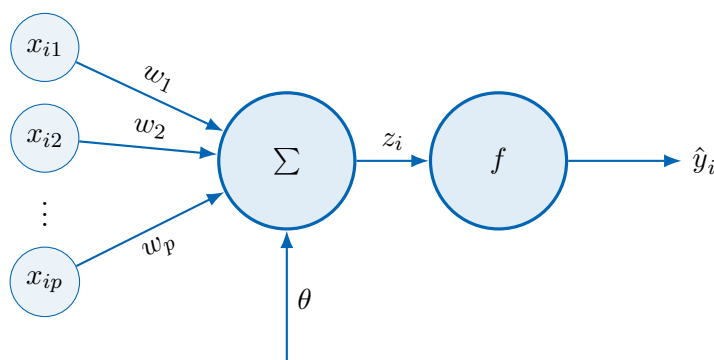


Figure 26: Formální neuron: vážený součet vstupů následovaný aktivační funkcí.

Volbou schodové aktivační funkce

$$f(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0 \end{cases}$$

získáme *perceptron*. Jeho rozhodovací hranice

$$\sum_{j=1}^p w_j x_j + \theta = 0$$

je nadrovina. Jeden perceptron tedy dokáže oddělit pouze lineárně oddělitelné třídy.

V hlubších sítích potřebujeme aktivační funkce, které dovolují postupně upravovat váhy pomocí derivací. Často používáme sigmoidální funkci zavedenou v definici sigmoidy nebo funkci

$$\text{ReLU}(z) = \max\{0, z\}.$$

Funkce ReLU ponechá kladný vstup beze změny a záporný nahradí nulou. Pro rozdělení výstupu mezi k tříd se používá funkce *softmax*. Pro vektor skóre $\mathbf{z} = (z_1, \dots, z_k) \in \mathbb{R}^k$ vrací

$$\sigma_i(\mathbf{z}) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}, \quad i \in \{1, \dots, k\}.$$

Všechny hodnoty $\sigma_i(\mathbf{z})$ jsou kladné a jejich součet je 1.

21.2 Síť a vrstvy

Definice 21.1. *Formální neuronová síť* je uspořádaná sedmice

$$\mathcal{N} = (V, E, I, O, w, \theta, f),$$

kde V je konečná množina neuronů, $E \subseteq V \times V$ je množina orientovaných spojů, $I \subseteq V$ je množina vstupních neuronů, $O \subseteq V$ je množina výstupních neuronů, $w: E \rightarrow \mathbb{R}$ přiřazuje spojům váhy, $\theta: V \setminus I \rightarrow \mathbb{R}$ přiřazuje nevstupním neuronům prahy a $f: \mathbb{R} \rightarrow \mathbb{R}$ je aktivační funkce.

Je-li graf (V, E) acyklický a spoje vedou pouze od dřívější vrstvy k pozdější, hovoříme o *dopředné neuronové síti*. Neurony rozdělíme do vstupní vrstvy, jedné nebo více skrytých vrstev a výstupní vrstvy.

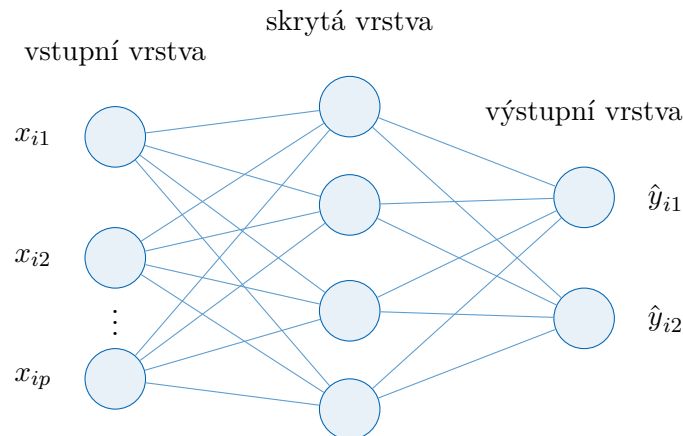


Figure 27: Vícevrstvý perceptron se vstupní, skrytou a výstupní vrstvou.

Počet skrytých vrstev označíme $L - 1$. Aktivace vstupní vrstvy pro objekt $\mathbf{x}_i$ tvoří vektor

$$\mathbf{a}^{(0)} = \mathbf{x}_i.$$

Má-li vrstva ℓ celkem n_ℓ neuronů, označíme její aktivace

$$\mathbf{a}^{(\ell)} = (a_1^{(\ell)}, \dots, a_{n_\ell}^{(\ell)}).$$

Váhu spoje z j -tého neuronu vrstvy ℓ do r -tého neuronu vrstvy $\ell + 1$ označíme

$$w_{jr}^{(\ell+1)}$$

a práh cílového neuronu $\theta_r^{(\ell+1)}$. Přenosovou funkci této vrstvy označíme $f^{(\ell+1)}$. Potom

$$a_r^{(\ell+1)} = f^{(\ell+1)} \left(\sum_{j=1}^{n_\ell} w_{jr}^{(\ell+1)} a_j^{(\ell)} + \theta_r^{(\ell+1)} \right).$$

Výpočet opakujeme od $\ell = 0$ až k výstupní vrstvě L . Tento postup nazýváme *dopředný průchod*.

Kdyby ve všech vrstvách byla aktivační funkce $f(z) = z$, celá síť by byla pouze složením lineárních funkcí, a tedy opět jedinou lineární funkcí. Nelineární aktivační funkce jsou proto nezbytné pro modelování složitějších hranic.

22 Učení neuronové sítě

Při učení s učitelem máme trénovací data

$$D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}.$$

Síť pro každý vstup provede dopředný průchod a vytvoří výstup $\hat{y}_i$. Ztrátová funkce porovná $\hat{y}_i$ se správnou hodnotou y_i . Pro regresi může jít o MSE, pro binární klasifikaci o křížovou entropii. Cílem je upravit všechny váhy a prahy tak, aby byla průměrná ztráta co nejmenší.

22.1 Zpětné šíření chyby

Derivaci jsme v předchozím výkladu popsali jako citlivost výstupu na malou změnu vstupu. Ve vícevrstvé síti potřebujeme určit citlivost ztráty na každou váhu. Změna váhy v první vrstvě nejprve změní aktivaci příslušného neuronu, ta ovlivní další vrstvu a teprve nakonec výstup a ztrátu.

Algoritmus *zpětného šíření chyby* (anglicky *backpropagation*) tyto závislosti počítá odzadu. Nejprve určí, jak změna výstupu ovlivní ztrátu. Potom postupuje od výstupní vrstvy ke vstupní a pro každý spoj násobí lokální citlivosti na navazující části výpočtu. Tím získá derivaci ztráty podle každé váhy a prahu.

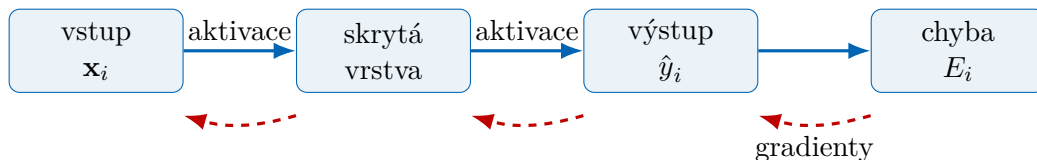


Figure 28: Dopředný průchod počítá výstup; zpětný průchod přenáší informaci o chybě.

Například pro jediný neuron

$$z = \sum_{j=1}^p w_j x_j + \theta, \quad \hat{y} = f(z), \quad E = \ell(y, \hat{y})$$

se citlivost ztráty na váhu w_j rozloží na tři navazující změny:

$$\frac{\partial E}{\partial w_j} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j} = \frac{\partial E}{\partial \hat{y}} \cdot f'(z) \cdot x_j.$$

Toto pravidlo se nazývá *řetězové pravidlo*. Není zde důležitá technika jeho důkazu, ale význam: celkový vliv získáme vynásobením vlivů jednotlivých navazujících kroků.

Při rychlosti učení $\eta > 0$ následně pro každou váhu provedeme aktualizaci

$$w \leftarrow w - \eta \frac{\partial E}{\partial w}$$

a obdobně upravíme prahy. Jeden úplný průchod trénovacími daty nazýváme *epocha*. V praxi data často rozdělíme na malé dávky a parametry aktualizujeme po každé dávce.

Při výpočtu gradientů je důležité použít aktivace z téhož dopředného průchodu. Kdybychom některou váhu změnili dříve, než dopočítáme gradienty ostatních vah, další derivace by již odpovídaly jiné síti. Proto nejprve získáme všechny potřebné gradienty a teprve potom provedeme aktualizaci parametrů.

Algoritmus 22.1: BACKPROPAGATION

Vstup: Trénovací množina D , síť s vahami a prahy, rychlost učení $\eta > 0$ a počet epoch T

pro $e \leftarrow 1, 2, \dots, T$ **opakuji**

pro každý objekt $(\mathbf{x}, y) \in D$ **opakuji**

 proved dopředný průchod a ulož aktivace všech vrstev

 vypočítej ztrátu $E = \ell(y, \hat{y})$

 od výstupní vrstvy ke vstupní vypočítej derivace $\frac{\partial E}{\partial w}$ a $\frac{\partial E}{\partial \theta}$

pro každý váha w a odpovídající práh θ **opakuji**

$w \leftarrow w - \eta \frac{\partial E}{\partial w}$

$\theta \leftarrow \theta - \eta \frac{\partial E}{\partial \theta}$

Výstup: Natrénované váhy a prahy sítě

22.2 Trénování a zobecnění

Malá trénovací ztráta sama o sobě nestačí. Síť se může naučit zvláštnosti konkrétních trénovacích dat a na nových objektech selhat. Tomuto jevu říkáme přeučení. Průběh proto sledujeme na validačních datech, která se nepoužívají k přímé aktualizaci vah.

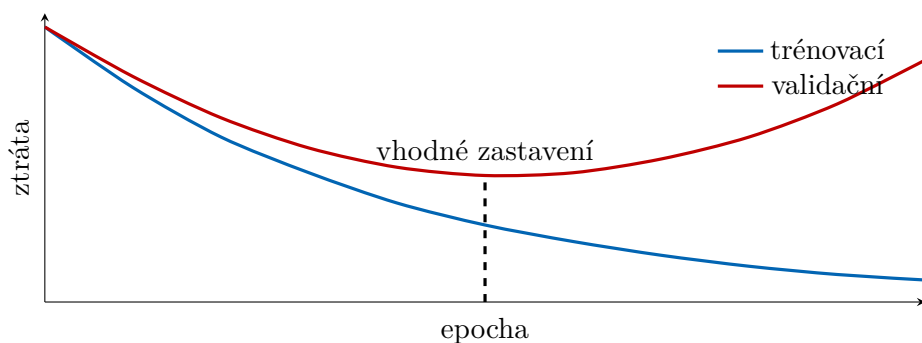


Figure 29: Při přeučení trénovací ztráta dále klesá, ale validační ztráta začne růst.

Přeučení omezujeme například menší sítí, větším množstvím trénovacích dat, penalizací příliš velkých vah nebo předčasným zastavením ve chvíli, kdy se validační ztráta přestane zlepšovat. Testovací data použijeme až na konci pro nezávislý odhad chování hotového modelu.

Opačným problémem je *nedoučení*: příliš malá síť, nevhodná rychlost učení nebo nedostatečný počet epoch nedokážou zachytit ani vztah přítomný v trénovacích datech. Trénovací i validační ztráta pak zůstávají vysoké. Je proto užitečné sledovat obě křivky současně, nikoli pouze jednu konečnou hodnotu.

Data se před učením obvykle promíchají a rozdělí do dávek. Promíchání omezuje vliv pořadí objektů a dávky představují kompromis mezi přesným, ale drahým gradientem celého souboru a rychlými, avšak kolísavými aktualizacemi po jednotlivých objektech.

22.3 Jednoduchá implementace

Program 22.1 implementuje síť se dvěma vstupy, jednou skrytou vrstvou a jedním sigmoidálním výstupem. Dopředný průchod si ukládá aktivace potřebné pro výpočet gradientů. Metoda `fit` potom provede zpětné šíření a gradientní sestup. Příklad učí logickou funkci XOR, kterou jediný perceptron kvůli nelineární oddělitelnosti nezvládne.

Váhy jsou na začátku zvoleny jako malá náhodná čísla. Kdyby všechny neurony jedné vrstvy začínaly se stejnými vahami, dostávaly by stejné gradienty a učily by se stále totéž. Náhodná inicializace tuto symetrii poruší a umožní jednotlivým neuronům zachytit různé vlastnosti vstupu.

Program 22.1: Dopředný průchod a zpětné šíření v malé neuronové síti.

```
1 from math import exp
2 from random import Random
3
4
5 def sigmoid(z):
6     return 1.0 / (1.0 + exp(-z))
7
8
9 class NeuralNetwork:
10     def __init__(self, input_size, hidden_size, seed=0):
11         random = Random(seed)
12         self.hidden_weights = [
13             random.uniform(-1.0, 1.0) for _ in range(input_size)
14             for _ in range(hidden_size)
15         ]
16         self.hidden_biases = [0.0] * hidden_size
17         self.output_weights = [
18             random.uniform(-1.0, 1.0) for _ in range(hidden_size)
19         ]
20         self.output_bias = 0.0
21
22     def forward(self, inputs):
23         hidden = [
24             sigmoid(sum(w * x for w, x in zip(weights, inputs)) + bias)
25             for weights, bias in zip(
26                 self.hidden_weights, self.hidden_biases
27             )
28         ]
29         output = sigmoid(
30             sum(w * a for w, a in zip(self.output_weights, hidden))
31             + self.output_bias
32         )
33         return hidden, output
34
35     def fit(self, inputs, targets, learning_rate=0.8, epochs=10000):
36         for _ in range(epochs):
37             for sample, target in zip(inputs, targets):
38                 hidden, output = self.forward(sample)
39
40                 # For sigmoid and binary cross-entropy:
41                 output_delta = output - target
42                 old_output_weights = self.output_weights[:]
43
44                 for j, activation in enumerate(hidden):
45                     self.output_weights[j] -= (
```

```

46         learning_rate * output_delta * activation
47     )
48     self.output_bias -= learning_rate * output_delta
49
50     for j, activation in enumerate(hidden):
51         hidden_delta = (
52             output_delta
53             * old_output_weights[j]
54             * activation
55             * (1.0 - activation)
56         )
57         for k, value in enumerate(sample):
58             self.hidden_weights[j][k] -= (
59                 learning_rate * hidden_delta * value
60             )
61         self.hidden_biases[j] -= learning_rate * hidden_delta
62
63     def predict(self, inputs):
64         return [self.forward(sample)[1] for sample in inputs]
65
66
67 xor_inputs = [(0.0, 0.0), (0.0, 1.0), (1.0, 0.0), (1.0, 1.0)]
68 xor_targets = [0.0, 1.0, 1.0, 0.0]
69
70 network = NeuralNetwork(input_size=2, hidden_size=3)
71 network.fit(xor_inputs, xor_targets)
72
73 for sample, output in zip(xor_inputs, network.predict(xor_inputs)):
74     print(sample, round(output, 3), int(output >= 0.5))

```

Ukázka vysvětluje princip, není však náhradou knihoven určených pro velké sítě. Ty používají maticové výpočty, automatické derivování, specializovaný hardware a další metody stabilizace učení.

Po spuštění se pravděpodobnosti pro vstupy XOR přiblíží požadovaným hodnotám 0,1,1,0. Kvůli náhodné inicializaci nemusí být každý běh číselně totožný, ale při vhodné rychlosti učení a počtu epoch by měl vést ke stejné klasifikaci.

23 Hopfieldův model

Dopředná síť neobsahuje orientovaný cyklus: informace postupuje od vstupu k výstupu a výpočet skončí. *Rekurentní síť* naopak dovoluje zpětné spoje. Její nový stav proto závisí na stavu předchozím a síť může uchovávat informaci v čase.

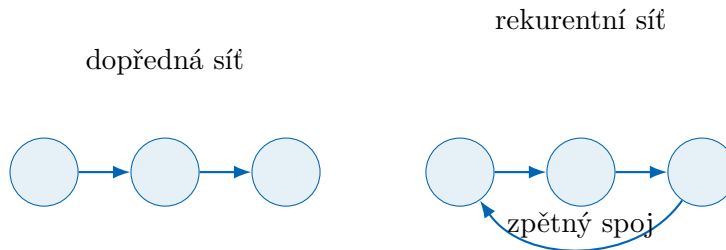


Figure 30: Rozdíl mezi dopřednou a rekurentní sítí.

23.1 Definice a dynamika

Definice 23.1. *Hopfieldova síť* s n neurony je dvojice

$$\mathcal{H}_n = (N, w),$$

kde $N = \{1, \dots, n\}$ je množina neuronů a

$$w: N \times N \rightarrow \mathbb{R}$$

je symetrická váhová funkce splňující

$$w(i, j) = w(j, i), \quad w(i, i) = 0.$$

Váhu $w(i, j)$ budeme krátce zapisovat w_{ij} . Každému neuronu i navíc přiřadíme práh $\theta_i \in \mathbb{R}$. Stav sítě v čase t je vektor

$$\mathbf{s}^{(t)} = (s_1^{(t)}, \dots, s_n^{(t)}) \in \{-1, 1\}^n.$$

Síť je plně propojená: každý neuron přijímá stav všech ostatních neuronů, vynásobí je vahami a vytvoří lokální pole

$$h_i^{(t)} = \sum_{j=1}^n w_{ij} s_j^{(t)} + \theta_i.$$

Při *asynchronní aktualizaci* vybereme jediný neuron i a nastavíme

$$s_i^{(t+1)} = \begin{cases} 1, & h_i^{(t)} > 0, \\ -1, & h_i^{(t)} < 0, \\ s_i^{(t)}, & h_i^{(t)} = 0. \end{cases}$$

Ostatní složky stavu zůstanou nezměněné. Neurony můžeme procházet v pevném nebo náhodném pořadí. Při synchronní aktualizaci bychom měnili všechny neurony najednou; ta však může mezi několika stavy kmitat.

Asynchronní pravidlo používá vždy nejnovější dostupný stav: po změně jednoho neuronu už následující neuron pracuje s touto novou hodnotou. Pořadí aktualizací může ovlivnit, do kterého stabilního stavu síť dospěje, ale při symetrických vahách neporuší vlastnost poklesu energie dokázanou níže.

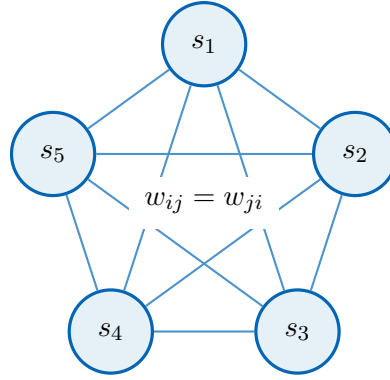
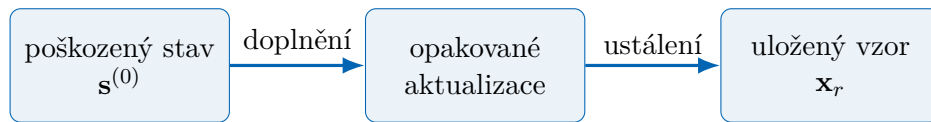


Figure 31: Hopfieldova síť má symetrické spoje a stav uložený v neuronech.

23.2 Asociativní paměť

Hopfieldova síť slouží jako *asociativní paměť*. Neuchovává záznam pod samostatnou adresou. Zapamatovaný vzor je stabilní stav sítě a je možné jej vybavit i z neúplné nebo mírně poškozené podoby. Opakované aktualizace postupně opravují jednotlivé složky, dokud síť nedospěje ke stabilnímu stavu.



nejbližší energetické minimum

Figure 32: Vybavení vzoru z jeho poškozené podoby.

Abychom odlišili počet neuronů od počtu ukládaných vzorů, zavedeme nové označení m pro počet vzorů. Necht

$$\mathbf{x}_1, \dots, \mathbf{x}_m \in \{-1, 1\}^n$$

jsou vzory určené k uložení. Jednoduché Hebbovo pravidlo nastaví pro $i \neq j$

$$w_{ij} = \frac{1}{n} \sum_{r=1}^m x_{ri} x_{rj}, \quad w_{ii} = 0,$$

kde x_{ri} je i -tá složka vzoru $\mathbf{x}_r$. Shodují-li se dvě složky ve většině vzorů, vznikne mezi nimi kladná váha; bývají-li opačné, váha je záporná.

Učení zde tedy neprobíhá opakovaným gradientním sestupem. Váhy vzniknou přímo jedním součtem přes ukládané vzory a potom se při vybavování již nemění. Opakovaně se mění pouze stav neuronů, dokud síť nenalezne stabilní konfiguraci.

23.3 Energetická funkce

Stavu $\mathbf{s} \in \{-1, 1\}^n$ přiřadíme energii

$$E(\mathbf{s}) = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} s_i s_j - \sum_{i=1}^n \theta_i s_i.$$

Změní-li se asynchronně pouze neuron i ze stavu s_i do stavu s'_i , symetrie vah dává

$$E(\mathbf{s}') - E(\mathbf{s}) = -(s'_i - s_i) \left(\sum_{j=1}^n w_{ij} s_j + \theta_i \right) = -(s'_i - s_i) h_i.$$

Když se neuron podle uvedeného pravidla skutečně změní, mají $s'_i - s_i$ a h_i stejné znaménko. Proto

$$E(\mathbf{s}') - E(\mathbf{s}) < 0.$$

Při každé změně tedy energie přísně klesne. Síť má jen 2^n stavů, a proto nemůže měnit stav donekonečna. Po konečném počtu změn dospěje k lokálnímu minimu energie, tedy ke stabilnímu stavu.

Tento důkaz se opírá o tři podstatné předpoklady: váhy jsou symetrické, na diagonále platí $w_{ii} = 0$ a v jednom kroku se mění pouze jeden neuron. U synchronní aktualizace se mohou změny různých neuronů navzájem ovlivnit a síť může přecházet mezi dvěma stavy, takže stejný argument nelze bez úpravy použít.

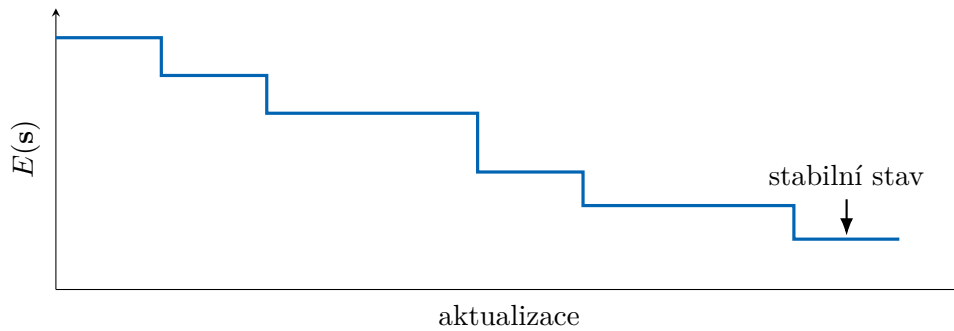


Figure 33: Aktualizace snižují energii, dokud síť nedospěje ke stabilnímu stavu.

23.4 Implementace a omezení

Program 23.1 vytvoří váhy Hebbovým pravidlem a poškozený vstup aktualizuje asynchronně. Nulové lokální pole ponechá původní stav neuronu, což odpovídá pravidlu použitému v důkazu poklesu energie.

Pseudokód odděluje dvě fáze. Procedura učení nejprve ze vzorů vytvoří symetrickou matici vah. Procedura vybavení potom přijme počáteční stav a provádí celé průchody neurony tak dlouho, dokud se v jednom průchodu žádný stav nezmění nebo dokud nedosáhne bezpečnostního limitu.

Algoritmus 23.1: HOPFIELD-RECALL

Vstup: Vzory $\mathbf{x}_1, \dots, \mathbf{x}_m \in \{-1, 1\}^n$, prahy θ_i , počáteční stav $\mathbf{s}$ a nejvyšší počet průchodů T

```
pro  $i \leftarrow 1, 2, \dots, n$  opakuj
    pro  $j \leftarrow 1, 2, \dots, n$  opakuj
        pokud  $i = j$  pak
             $w_{ij} \leftarrow 0$ 
        pokud  $i \neq j$  pak
             $w_{ij} \leftarrow \frac{1}{n} \sum_{q=1}^m x_{qi} x_{qj}$ 
pro  $r \leftarrow 1, 2, \dots, T$  opakuj
    zmena  $\leftarrow 0$ 
    pro  $i \leftarrow 1, 2, \dots, n$  opakuj
         $h_i \leftarrow \sum_{j=1}^n w_{ij} s_j + \theta_i$ 
        pokud  $h_i > 0$  pak
             $s'_i \leftarrow 1$ 
        pokud  $h_i < 0$  pak
             $s'_i \leftarrow -1$ 
        pokud  $h_i = 0$  pak
             $s'_i \leftarrow s_i$ 
        pokud  $s'_i \neq s_i$  pak
             $s_i \leftarrow s'_i$ , zmena  $\leftarrow 1$ 
    pokud zmena = 0 pak vrať  $\mathbf{s}$ 
```

Výstup: Poslední stav $\mathbf{s}$

Program 23.1: Hopfieldova asociativní paměť.

```
1 class HopfieldNetwork:
2     def __init__(self, size):
3         self.size = size
4         self.weights = [[0.0] * size for _ in range(size)]
5
6     def fit(self, patterns):
7         for i in range(self.size):
8             for j in range(self.size):
9                 if i != j:
10                    self.weights[i][j] = sum(
11                        pattern[i] * pattern[j] for pattern in patterns
12                    ) / self.size
13
14     def recall(self, damaged_pattern, max_sweeps=20):
15         state = damaged_pattern[:]
16         for _ in range(max_sweeps):
17             changed = False
18             for i in range(self.size):
19                 field = sum(
20                     self.weights[i][j] * state[j]
21                     for j in range(self.size)
22                 )
23                 new_value = 1 if field > 0 else -1 if field < 0 else state[i]
24                 if new_value != state[i]:
25                     state[i] = new_value
26                     changed = True
27             if not changed:
28                 break
```

```

29         return state
30
31
32 patterns = [
33     [1, 1, 1, -1, -1, -1, 1, 1, 1],
34     [1, -1, 1, 1, -1, 1, 1, -1, 1],
35 ]
36
37 network = HopfieldNetwork(size=9)
38 network.fit(patterns)
39
40 damaged = [1, 1, -1, -1, -1, -1, 1, 1, 1]
41 print(network.recall(damaged))

```

Kapacita Hopfieldovy sítě je omezená. Při příliš velkém počtu uložených nebo silně podobných vzorů se jejich příspěvky ve vahách ruší. Síť potom může skončit ve špatném uloženém vzoru nebo v nežádoucím stabilním stavu, který nebyl součástí trénovacích dat. Model je proto užitečný hlavně jako názorný příklad rekurentní sítě, asociativní paměti a učení bez učitele.

Kód záměrně používá malé binární vzory, aby bylo možné sledovat každý krok. U obrázků bychom pixely nejprve převedli na hodnoty -1 a 1 , obraz zploštili do vektoru a po vybavení jej opět složili do původního tvaru. Princip vah i aktualizací by zůstal beze změny.

Poznámka 23.2. John Hopfield publikoval známý model asociativní paměti roku 1982. Jeho práce propojila dřívější myšlenky rekurentních sítí s energetickou funkcí známou ze statistické fyziky a výrazně přispěla k obnovenému zájmu o neuronové sítě.